Imperial College London
Department of Computing

Using Database Schemas and Conceptual Modelling to Guide Data Visualisation

Author: David Bull

Supervisor: Dr Peter McBrien

Submitted in partial fulfilment of the requirements for the MSc Degree in Computing of Imperial College London

September 2023

Abstract

This report outlines the design, implementation and evaluation of an application for schema-driven data visualisation in the analysis of relational databases. The application provides a proof-of-concept for research already under way at Imperial College London, which proposes that the underlying relationships in relational database schemas can be used to improve guided analytics through visualisation recommendation, when compared to the current state of the art.

Acknowledgments

I would like to thank my supervisor, Dr Peter McBrien, for investing the time to provide invaluable advice and support throughout the project.

I would also like to thank the developers of the Amazing-ER tool, created by previous students at Imperial College London, for providing the foundations for some of the project's core functionality.

Finally, I would like to thank all those who gave up their time to help test and refine the tool during its development.

Contents

Overview

1.1 Project Motivation

Modern tools for data visualisation are largely designed to work with single, tabular datasets as their primary input. As a result, the benefits of analytical enquiry through visual representation and interrogation of data can only be realised *after* an initial process to create the dataset for analysis through data audit, data integration and feature selection.

In practice, gaining a full appreciation of data relating to a domain will often begin with large, relational databases – particularly in the context of business or public services where customer relationship management (CRM), people management, finance management and electronic point of sale (EPOS) systems are core to most organisations.

The ever-increasing quantity of data stored in these systems, along with the fact that normalised data does not lend itself to quick, human interpretation, means that analysts are more likely to miss important features in their data or at least make sub-optimal choices in the data used to represent key features in analytical datasets [1, 2].

To assist in the challenges posed by the quantity of data available, a number of tools have been developed to provide guidance through the full or partial automation of data analysis and visualisation tasks [3, 2, 4, 5, 6, 7]. A core feature of the approach taken by many of these tools is to enumerate all possible visualisations based on partial specification by a user, and to select visualisations for presentation to users by rankings based on certain criteria.

Whilst these approaches have made great advances in helping analysts to under-

stand data*sets*, we believe that there are further advances that can be made in helping analysts to understand their relational data *schemas*. To our knowledge, no approaches to visualisation recommendation leverage the underlying structure of relational datasources. By doing so, we believe that assistance can be provided to analysts from the outset of an analytical workflow, whilst the search space for the enumeration of candidate visualisations in current visualisation recommendation tools can be reduced.

1.2 Project Objectives

Research under way at Imperial College has already begun to explore the best approaches to data visualisation based on conceptual modelling of relational datasources, focusing on the algorithms that can be applied to database systems to map schema information to the visual domain [1]. This project aims to advance this research by building an end-to-end implementation of these principles, including a visualisation layer.

Our belief is that a full-stack application will help to accelerate research in this area by facilitating faster iteration and testing of hypotheses. At the same time certain challenges may only be solved through the provision of a user interface – for example, in narrowing the search space of candidate visualisations through user interaction rather than bottom-up computation.

In support of these objectives, the project also contributes a Java library for constructing visualisations in the Vega specification language. This was developed to support the immediate needs of the project, as the project identified Java as the best back-end technology for the project goals, as well as identifying Vega as a strong candidate for visualisation specification.

In addition to the immediate needs of the project, though, we believe that the provision of Java development environment for Vega will reduce the barrier to entry for database researchers and those with a data engineering skillset, who are likely to be familiar with lower-level statically typed languages.

Background

This project rests on three key areas of research, which are reviewed in detail in this section.

- Visualisation specification and the Grammar of Graphics
- Approaches to automated data visualisation
- Data visualisation through conceptual modelling of relational databases

2.1 Visualisation Specification & the Grammar of Graphics

2.1.1 The Grammar of Graphics

The central argument of Leland Wilkinson's *Grammar of Graphics* [8] is that visualisations of data should be considered a composition of hierarchical elements for which definition and interaction follow a systematic grammar, rather than thinking of visualisations as monolithic objects that fit into a fixed taxonomy.

The monolithic taxonomy of visualisations offers users set 'chart types' to represent data, such as bar charts, scatter plots, line charts or pie charts. This approach tends to constrain users to a smaller set of 'out-of-the-box' visualisations, and often offers limited functionality in terms of iteratively updating and customising the components of the graphic [9]. From a computational perspective this also leads to the re-creation of all graphical components from scratch as different user selections are made, rather than varying specific components of the graphical specification and

allowing these amendments to drive what is rendered on screen [8].

As an alternative, Wilkinson sets out a concise set of elements that can be varied programatically, in isolation, and combined to produce most, if not all, visualisations:

- **Variables - Extracting relevant information from a dataset:**
  This includes transformations of data - which may be mathematical (e.g. logarithmic or exponential transformations), statistical (e.g. mean, rank, sort) or multivariate (e.g. group, sum, product). It is also important to categorise resulting variables, according to whether they are categorical or continuous, for example, to direct later mappings and transformations [8, pp.55-61].

- **Algebra - Defining operations and rules for combination:**
  This defines how variables and chart components may be combined and provides a structured parse tree of operations that can be translated programmatically. At the simplest level Wilkinson specifies the cross, blend and nest operations, akin to cross joins, unions and groups in SQL or relational algebra [8, pp.63-83]. These algebras have been extended and refined in subsequent work [10, 11].

- **Scales - Mapping sets of variables to dimensions:**
  Scales define units of measurement and level of detail and are a key driver of perception and interpretation of resulting graphics [12]. Scales play a central role in transforming variables to a visual domain, for example by representing ordinal data on an implied, ordered numerical scale - or by transforming from a linear scale to a logarithmic scale to improve visual clarity [8, pp.85-109].

- **Statistics - Providing methods to alter the geometry of a graphic:**
  Under Wilkinson's definition, statistics are graphing functions which are - in contrast to traditional approaches to visualisation - separated from the data to which they are applied, and separated from the visual graphic that they are associated with. Thus statics can be altered and combined in isolation from the rest of the components in a graphic.

  Wilkinson proposed five key types of statistical methods: Binning (i.e. partitioning or tiling a space), Summary (e.g. mean, median, mode), Regions (e.g. intervals, ranges, bounds), Smoothing (e.g. quadratic smoothing or heatmap density) and Linking (e.g. defining sequences and neighbours) [8, pp.111-154].

- **Geometry - Producing points which map to a set of coordinates:**
  These may be simple functions (e.g. to create a point, line, area, interval or path), they may be partitions of the visualisation space (e.g. polygons that

denote countries on maps) or networks (e.g. the edges between nodes).

Wilkinson also defines a set of collision modifiers that avoid visual obstruction between points in a geometric space - namely stacking (as in the case of a stacked bar chart), dodging (plotting the overlapping point in the closest area of free space) and jittering (adding an extra dimension to the visualisation and spreading points across them) [8, pp.155-178].

- **Coordinates - Defining the plane on which points are mapped:**
  For example, the same set of points may be mapped to a cartesian or a polar coordinate system, to produce different visual results (see Figure 2.1) or polygons may be projected onto a different cartographic plane, as in the case of geographic maps. Separating coordinates also allows plane transformations within a given geometry, such as reflection and rotation [8, pp.179-254].

  In the example shown in Figure 2.1, for instance, Wilkinson demonstrates that rather than specifying two separate visualisations of type 'bar chart' and 'nightingale rose chart', the specification of the two charts is identical apart from the addition of polar coordinates ('COORD: polar') to produce the nightingale rose.

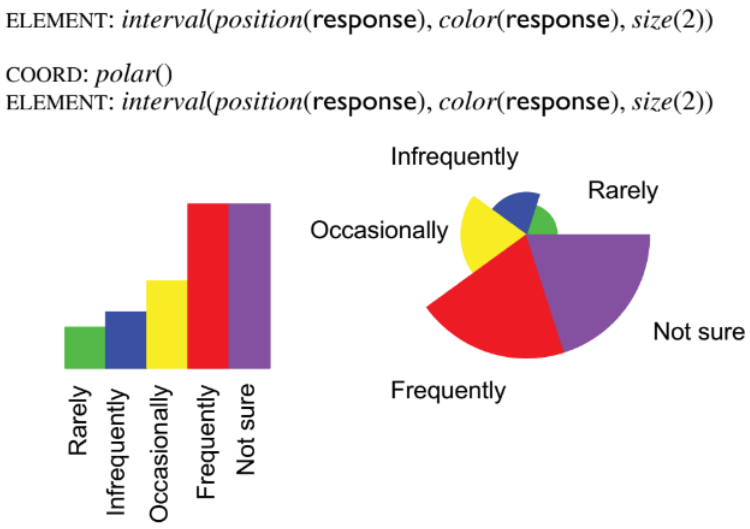


Figure 2.1: *Taken from the Grammar of Graphics, p.211 - demonstrating a single transformation of plane between a bar chart and a 'nightingale rose' chart [8].*

- **Aesthetics - Assigning dimensions to visual channels:**
  Wilkinson builds on Bertin's Semiology of Graphics [13] to set out five key characteristics of visual attributes in terms of Form (e.g. position, size, shape, rotation), Surface (e.g. colour and texture), Motion (e.g. direction and speed),

Text (in terms of labels and legends) and even Sound (varied through tone, volume, rhythm and voice).

Wilkinson also lays out a number of rules for visual attributes, namely that they must be capable of representing both continuous and categorical variables (which may mean some transformation, for example conforming colour to a one dimensional scale to represent continous variables), that visual attributes must represent a distinct, drawable feature within a rendering system, and that assignment of dimensions to visual attributes should allow clear interpretation and distinction by viewers, according to established theory [14, 15, 13].

The innovation in this approach was to see that the visual representation of a graphic could be defined and refined by variation of these components. This meant that users could quickly and iteratively update visualisations by focusing on *aspects* of their data and their visualisations for investigation, rather than trying to fit the data into a prescribed list of visual forms. This meant that rendering and re-rendering of visualisations on screen could be done efficiently - by recalculating only those elements of the hierarchical component tree affected by a change and attaching event listeners to specific aspects of the visualisation rather than the graphic as a whole. Transitions between entirely different visual representations could now be achieved through varying a small element of a visual specification, meaning that enumerating different visualisations could be done quickly and programatically.

Wilkinson's approach underpins most visualisation specification languages used in data visualisation tools today and will form a key part of the way that visualisations are constructed in this project.

2.1.2 Visualisation Specification

Visualisation specification languages provide a structured, declarative framework for the creation of visualisations. Based on grammars of graphics such as Wilkinson's, they provide rules, syntax and components for the creation of visualisations.

Significantly, as declarative languages, the visualisation specification is separated from what Wilkinson calls the 'Assembly' phase, which entails combination of the specified components and compilation into a format that can be rendered, and the 'Display' phase, which uses OS primitives and other specialised software to render visualisations to the screen [8, 9].

Specification languages can be broadly split between 'high-level' specification, employed in tools such as ggplot2 [16, 17, 18], Vega-Lite [19] and VizQL (underpinning Tableau) [10, 7] and 'low-level' specification, employed in tools such as Vega [20] and D3 (*Data Driven Documents*) [21]. Low-level specification means a low-level API, giving fine-grained control over all aspects of the underlying grammar and all visualisation components. By contrast, high-level specification provides a more concise specification language by abstracting certain elements of visualisation construction, or by limiting functionality.

low-level是命令式语言，详细说明坐标轴怎么画
high-level是声明式语言，仅说明映射关系，系统自己渲染坐标轴

For example, Vega (the lower-level counterpart to Vega-Lite) offers the custom definition of visual 'marks' (akin to Wilkinson's *geometry*) and the custom definition of the visual channels through which they are encoded (akin to Wilkinson's *aesthetics*) where Vega-lite supports a fixed (yet still fairly comprehensive) set of mark and encoding types [22, 23].

It is generally considered that lower-level specification languages allow for greater customization and control over behaviour, at the cost of higher complexity and much more verbose specification. The creators of Vega characterise high-level specification languages as favoring "conciseness over expressiveness" – making high-level specification better for rapid, exploratory visualisation with a shallower learning curve and low-level specification more appropriate for in-depth explanatory visualisation [19].

2.1.3 Open-source Visualisation Ecosystems

A central design decision for this project was to decide on the languages and tools to use in constructing and displaying data visualisations.

The term **ecosystem** is used here to highlight the fact that most options include a combination of Wilkinson's three stages - *Specification*, *Assembly* and *Display*. For example, most provide a language for grammatical visual specification alongside a set of tools to compile and render the resulting visualisations.

In order to differentiate between these possible ecosystems, an evaluation is included below based on selected criteria, which is summarised in Figure 2.2.

Only open-source tools are considered here to ensure accessibility and longevity for the project – pointing to six potential visualisation ecosystems: *Vega, Vega-Lite,*

Ant-V, D3, Google Charts and ggplot2.

Out-of-the box visualisation support

Ant-V and *Google Charts* are the only options that support out-of-the-box visualisations – by which a dataset can be passed in to create common chart types. *Ant-V* has the greatest range of available visualisations by a considerable margin – being the only tool to support chord diagrams out-of-the-box, for example.

Vega, *Vega-Lite*, *ggplot2* and *D3* require that charts are constructed from their constituent parts (e.g. constructing scales, axes and marks etc.), though common data transformations are supported – for example, *D3*'s chord transform and *Vega*'s treemap transform.

Support for specification through visualisation grammar

Google Charts is the only option that does not support this approach to visualisation, though it has recently added support for *Vega* specification within *Google Charts* to allow fully custom visualisations. *Vega*, *Vega-Lite*, *ggplot2* and *D3* have this approach to visualisation specification at their core and are designed primarily as grammars for visualisation specification.

Vega-Lite is an intentionally limited subset of *Vega*, which does not support the creation of a number of more complex visualisations – including Treemaps, Word Clouds, Chord Diagrams, Network Diagrams and Sankey Diagrams. *Vega-Lite* instead focuses on brevity of specification and ease of building interactive charts.

Ant-V stands out in providing both a visualisation grammar for custom visualisation specification as well as providing a large selection of pre-built visualisations that are ready to use. It is the case, however, that *Vega*, *D3* and *ggplot2* provide more faithful and comprehensive grammars when compared to the original *Grammar of Graphics*.

Language support

Vega and *Vega-lite* are the only options to support specification in multiple languages. A core design principle of *Vega* is to provide a declarative JSON interface – meaning that specifications can be designed in any language that can produce a JSON specification.

By contrast, *ggplot2* is designed only for *R*, whilst all other options exist firmly in the realm of web development, and therefore require JavaScript/Typescript knowledge

for their implementation. Of these *D3* requires the deepest knowledge of JavaScript, whilst *Ant-V* and *Google Charts* support a slightly simpler implementation.

Support for interactive and responsive visualisation

Interactive features include functionality such as tooltips, zooming and drill-downs, whilst responsive features include functionality such as the automatic resizing and re-orientation of text labels, chart elements and tickmark spacing in response to screen size, as well as the avoidance of visual clutter such as label overlap.

Ant-V and *Google Charts* have the best support in this respect, with many features included as standard without configuration. *Ant-V* also includes considerable customisation options, including full JavaScript specification of chart elements such as tooltips, axes and labels.

At the other end of the spectrum, *ggplot2* is designed primarily for static, offline environments, meaning that the core grammar does not consider interactivity and responsiveness.

Dataset format

Google Charts expects a data format that is not consistent with all the other available tools, in that it accepts an array of arrays, with each row of values represented by an array, with the first array containing field names.

By contrast, all the other tools evaluated expect an array of JSON objects, with a JSON object for each row containing key-value pairs of field names and field values. This puts *Google Charts* at a disadvantage in any application that may want to support multiple visualisation packages — as this would require storing multiple representations of the data, as well as processing of datasets to convert between formats (which would have linear complexity in relation to the size of the analytical dataset).

D3, *Vega* and *Vega-Lite* also have additional support for loading data directly from URLs pointing to CSV, JSON and XML files.

Whilst *ggplot2* technically only accepts data in the format of an *R* dataframe, this is mitigated by the fact that *R*'s mature data ingestion capabilities can quickly handle this transformation from most formats.

	Out-of-the box visualisation	*Visualisation grammar support*	*Language Support*	*Interactive/Responsive Visualisation*	*Dataset format (inter-operability)*	*Total*
Vega	1	5	5	1	5	17
Ant-V	5	3	1	5	3	17
Vega-Lite	1	3	5	1	5	15
D3	1	5	1	3	5	15
Google Charts	3	1	1	5	1	11
ggplot2	1	5	1	1	3	11

Figure 2.2: *A comparison of leading visualisation ecosystems. Each category was given a 'T-shirt size' rating of high, medium or low based on the above assessment, which was translated to a numerical score of 1,3 or 5 to give an indicative ranking.*

Based on this assessment, *Vega* and *Ant-V* were chosen as the focus for developments during this project.

2.2 Approaches to Automated Data Visualisation

The APT system devised by Jock Mackinlay [24] was a seminal work in the automated visualisation of data. APT sought to codify graphic design criteria and best practice [13] into a automated presentation tool using a formal algebra which Mackinlay referred to as 'sentences' of 'graphical language'.

A key contribution of the APT system, which is a core part of many tools used today, was the concept of expressiveness and effectiveness criteria as a means to rank and evaluate visualisations. '*Expressiveness*' refers to the ability of a visualisation to fully encode the 'facts' of a dataset without impying incorrect or extraneous information. For example, a visualisation specification would not be expressive if it sought to encode mutliple dimensions to the same visual channel. '*Effectiveness*' refers to the extent to which a visualisation conveys the intended meaning - which rests upon theories of design and human perception [14]. A visualisation would be considered less effective, for example, if it sought to distinguish unordered, nominal categories by their size (as this encoding would most likely imply ordering to the observer) [24].

The Polaris system (which later became Tableau) built on APT, extending the system's algebra, incorporating the approaches of Wilkinson's *Grammar of Graphics* and allowing visual specifications to be more easily compiled into database queries. The system, as Tableau, also introduced *Show Me* [7], an approach to the automatic generation of visualisation recommendations. *Show Me* works partly through the use of heuristics to define the appropriate mark types based on user selection. For

example by automatically selecting a geographic representation for spatial data, or by discounting categorical encodings (such as shape or colour) where there are not a sufficient number of shapes to cover the full cardinality of the selected field. The recommended charts produced by *Show Me* are based firstly on a mapping of chart types to field types (i.e. mapping chart types to the minimal set of data types required to produce them, namely categorical, quantitative, date and geographical data types), and secondly ranking the different chart types based on visual effectiveness criteria.

Voyager, and subsequently *Voyager 2*, sought to focus on *breadth-first* exploratory data analysis by generating a large number of possible visualisations, presenting them to users in a faceted browser and allowing analysis to proceed by gradual refinement through a cycle of user input and visualisation recommendation [9, 2]. Voyager uses its *CompassQL* query language and *Vega-Lite* visualisation specification to quickly enumerate candidate visualisations first by iterating through different data field combinations, then through different mark representations of the data (e.g. point vs. line vs. area) and through different encodings of the data (e.g. colour vs. shape vs. size).

The two key challenges faced by any of these systems are: (1) enumerating candidate visualisations in a way that is efficient and feasible in the context of dynamic, interactive systems – avoiding what the creators of Voyager call "combinatorial explosion" [9], and (2) ranking and prioritising the visualisations that are produced in order to narrow down a digestible number of recommendations to users. Systems such as Tableau employ partial specification of the data by a user combined with effectiveness and expressiveness rankings [7], where systems such as SeeDB [25, 4], Autovis [3] and Scagnostics [26] have taken a data driven approach - choosing to recommend visualisations based on analysis of a dataset in its entirety, prioritising visualisations based on whether they demonstrate features of interest - such as outliers or strong correlations. Voyager 2 takes a hybrid approach by enumerating based on partial specification, data features and visual encodings - reducing the search space by prioritising simpler chart types suitable for trellis displays, removing similar visualisations, conducting phased enumeration based on user input and also layering on expressiveness and effectiveness metrics [2].

This project intends to contribute to this space by exploring conceptual modelling of relational database schemas as a means to reduce the search space for visualisation enumeration and to prioritise visualisation recommendations.

2.3 Data Visualisation of Relational Databases

McBrien and Poulovassilis outline an approach to visualisation recommendation that not only considers the types of fields in a dataset, but also the relationships between fields in the underlying database schema.

Motivating example

Figure 2.3 shows the relationship between the *waterbody* and *island* entities in the Mondial database [27]. Using a schema-driven approach, and therefore using the relationship between entities to inform visualisation recommendation, a one-to-many relationship identifies a hierarchical relationship between parents and children. In this case, one waterbody contains many islands.

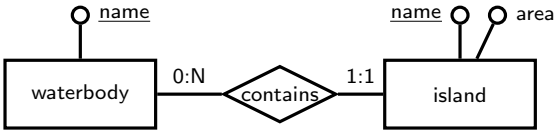


Figure 2.3: *ER schema of the 'island' and 'waterbody' entities in the Mondial database*

Where other visualisation tools do not make use of this underlying schema information, its incorporation into visualisation recommendation means that a hierarchical visualisation type, such as the treemap shown in Figure 2.4, can be selected as the most appropriate visualisation of the data without any user interaction beyond attribute selection.

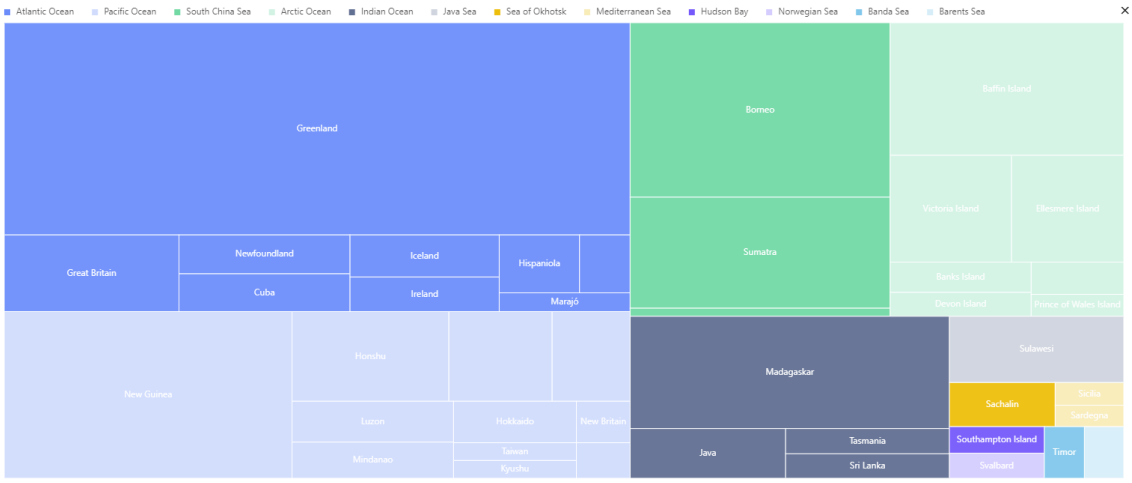
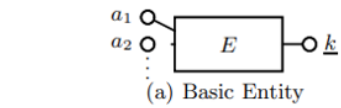


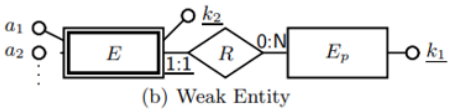
Figure 2.4: *A treemap visualising the attributes shown in Figure 2.3*

In this way both attribute values and the participation of entities in relationships can be used to drive visualisation recommendation. The initial mappings proposed are shown below in Figure 2.5. By providing a set of mappings between schemas and visualisations, the approach also offers a new approach to the enumeration of candidate visualisations, in that common transformations (e.g. pivoting and denormalisation) can be applied to raw relational schemas to determine further appropriate visual encodings [1].



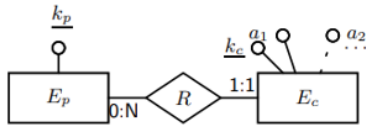
(a) Basic Entity

Basic Entity Visualisations			
Name	$\|k\|$	mandatory	optional
Bar Chart	1..100	a_1 scalar	-
Calendar	1..*	a_1 temporal scalar	a_2 colour
Scatter Diagrams	1..*	a_1, a_2 scalar	a_3 colour
Bubble Charts	1..*	a_1, a_2, a_3 scalar	a_4 colour
Choropleth Maps	1..*	k geographical, a_1 colour	-
Word Clouds	1..*	k lexical, a_1 scalar	a_2 colour



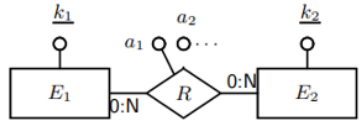
(b) Weak Entity

Weak Entity Visualisations					
Name	$\|k_1\|$	$\|k_2\|$	complete	mandatory	optional
Line Chart	1..20	1..*	no	k_2, a_1 scalar	a_2 scalar
Stacked Bar Chart	1..20	1..20	yes	a_1 scalar	-
Grouped Bar Chart	1..20	1..20	no	a_1 scalar	-
Spider Chart	3..10	1..20	yes	a_1 scalar	-



(c) One-Many Relationship (Hierarchy)

One-many relationships				
Name	$\|k_1\|$	$\|k_2\|$ per k_1	mandatory	optional
Tree Map	1..100	1..100	a_1 scalar	a_2 colour
Hierarchy Tree	1..100	1..100	-	a_1 colour
Circle Packing	1..100	1..100	a_1 scalar	a_2 colour



(d) Many-Many Relationship

Many-many relationships					
Name	$\|k_1\|$	$\|k_2\|$	reflexive	mandatory	optional
Sankey	1..20	1..20	no	a_1 scalar	a_2 colour
Chord	1..100	1..100	yes	a_1 scalar	a_2 colour

Figure 2.5: *Figures collected from 'Towards Data Visualisation based on Conceptual Modelling and Schema Transformations', McBrien and Poulovassilis, 2018) [1].*

A key benefit of this approach is to allow analysts to begin exploratory data visualisation without any abstraction or preparation of their source business systems. At the same time, this approach provides a novel approach to ruling *out* inappropriate visualisation patterns as a way to reduce the search space of candidate visualisations, which is an approach that could be used to enhance the methods already employed in tools such as *Voyager 2*. For example, using this approach can quickly ascertain that a treemap should not be prioritised as a recommendation for data with a many-to-many relationship, as such a relationship will not fit cleanly into a hierarchical form.

The effectiveness of this approach relies on the existence of a well structured database with clearly defined constraints, which may not always be the case in all organisations. Whilst initial development will assume the existence of these dependencies, it is also the case that, if necessary, techniques exist to reverse engineer many of the required structure and constraints [28, 29, 30, 31, 32].

2.4 Contribution to the Current State of the Art

We believe that this approach will advance the current state of the art, which is demonstrated here through comparison to three leading data visualisation tools.

It should be emphasised, however, that the purpose of this project is not to develop a full replacement for these tools, but rather a proof of concept to demonstrate missing features that can advance the field as a whole.

2.4.1 Tableau and Power BI

Tableau and Power BI are the market-leading proprietary data visualisation and analysis tools. They are sophisticated, enterprise-level analytics tools that provide extensive data ingestion capabilities for a range of data formats and system types, along with data preparation and transformation capabilities, big data processing engines, user administration and security management.

The key areas where this project will seek to contribute features not supported by Tableau and Power BI are:

- **The use of database schema information to support analytics**. Both Tableau and Power BI provide tools to access and combine data sources, but neither makes the underlying database schema information, such as primary and foreign keys, explicit to users nor make use of this information to aid users in the construction of datasets.
- **Visualisation recommendation for complex relationships**. Whilst Tableau does support visualisation recommendation (see Section 2.2) it does not support the recommendation of more complex chart types, such as Chord and Sankey Diagrams. This means that recommended visualisations of many-to-many relationships can be inappropriate.

Conversely, though Power BI can support complex visualisations more easily through JavaScript extensions, it does not provide any visualisation recommendation, instead leaving design choices to the user.

- **Re-enforcing visualisation best practice**. Tableau's design means that recommended visualisations are highlighted, but no explicit guidance is given to the user on visualisation types that should be *avoided*, which can lead to misleading and ineffective design choices by users. This project's recommendation approach provides greater clarity in that respect.

2.4.2 Voyager 2

Voyager 2 is most similar to this project in its design and intention – as an open-source project using Vega-Lite to provide visualisation recommendations to aid analytical enquiry.

As described in Section 2.2, Voyager 2 focuses on providing an iterative, exploratory analytical workflow – using broad visualisation recommendations as a way to narrow down analytical enquiries.

The key differences in the goals of this project compared to Voyager 2 are:

- **Focus on exploratory depth rather than exploratory breadth**. The stated aim of Voyager 2 is to provide a breadth of potential data combinations via simple, compact visualisations. Visual representations of the data are therefore simple by design, which may lead to sub-optimal visualisations of some datasets. This approach is not necessarily better or worse than prioritising exploratory depth. Instead we aim to contribute an alternative design that may be better suited to some analytical use cases.
- **Support for analysis of full database systems, rather than individual datasets**. Voyager 2's focus is on combining data attributes primarily from single datasets, without accounting for the relationship between attributes in the context of database schemas. By focusing on this element of the data we provide a way to supplement recommendations, whilst also limiting the search space of potential visualisations - which can aid a breadth-first exploratory approach such as that used in Voyager 2.

Schema-driven Visualisation Recommendation

3.1 Application Design

The key objective of the application is to provide a proof of concept via an end-to-end implementation of the visualisation recommendation approach outlined in *Towards Data Visualisation Based on Conceptual Modelling* [1].

3.1.1 Design Requirements

The design specification against which the application was built specifies the following goals:

- Identification and visualisation of basic entity, weak-entity and one-many relationships
- Identification and visualisation of 'is-a', subset relationships
- Complex visualisations of many-many relationships (such as chord and sankey diagrams) in contrast to current market leading tools.
- Guidance on the appropriate data cardinality for different visualisation types, also giving users the ability to adjust these thresholds where appropriate
- Automated SQL query generation for the relationship types specified above

- Presentation of underlying schema patterns and data relationships in the user interface, to users to allow users to better understand the underlying data system and the appropriate visualisation types
- Visualisation recommendations to guide users towards the most appropriate visualisation types, including signposting of visualisations that are specifically *not* recommended.

3.1.2 Target audience

The application was designed to present minimal barriers to entry to allow its use by non-technical business users as well as analysts. For this reason a simpler point-and-click interface is provided for interaction with application - allowing any user to fulfill simple use cases.

The application is also designed to support more advanced users, however, meaning that features are included to support more in-depth understanding of the underlying data and relationships, as well as the direct manipulation of SQL queries for data analysis.

3.1.3 High-Level Architecture

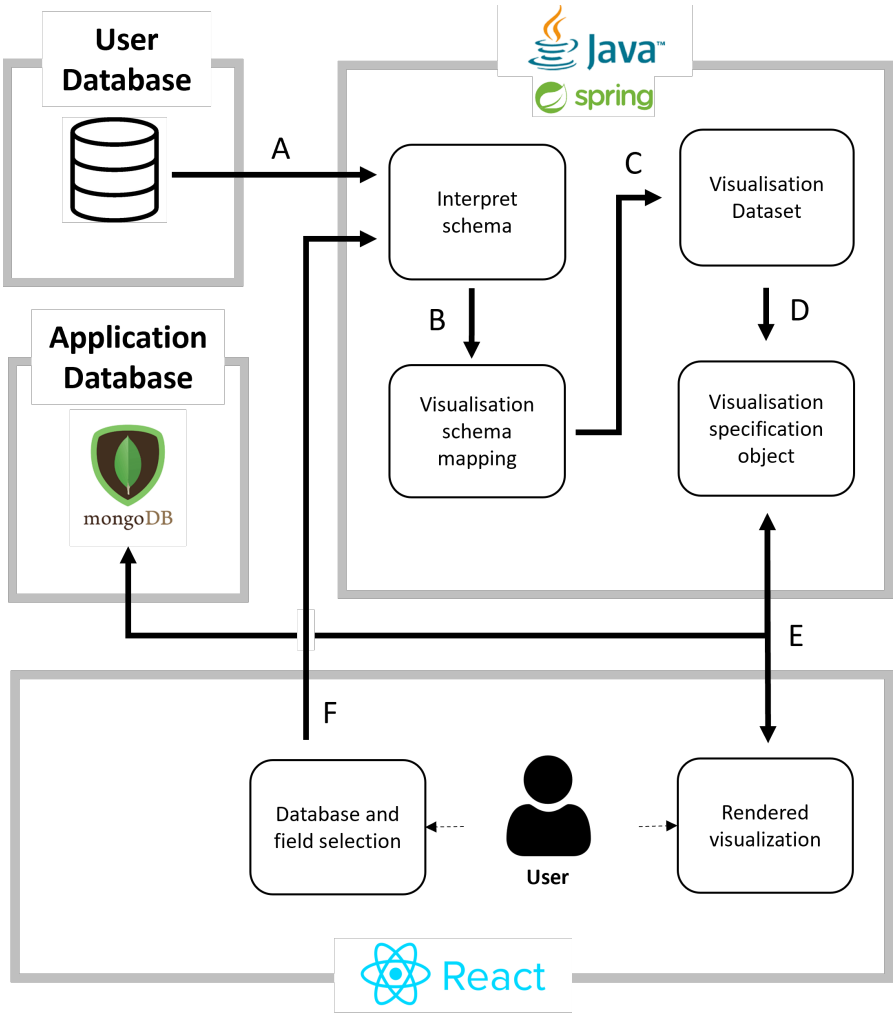


Figure 3.1: *The high-level application architecture for this project*

The labelled steps in the design shown in Figure 3.1 are as follows:

A Extract the schema and constraints of a specified database using JDBC.

B Map schema information to visualisation patterns using methods devised through prior research at Imperial College London.

C Construct the required dataset implied by the visualisation schema pattern and transform it to JSON.

D Define classes and interfaces to allow visualisation specifications to be built programmatically according to object orientated design principles.

E Specification objects can be marshalled and un-marshalled using the Jackson library to create JSON objects to be passed to the front-end and/or persisted in MongoDB. Communication between the back-end and front-end is achieved through Spring Boot and Axios.

F User interaction on the front-end will determine the database fields used in visualisation specification, facilitating guided exploration of the data, whilst also limiting the search space for back-end computation.

3.1.4 User Journey and High-Level Design

The following section describes a user journey through the application, along with a high-level overview of the way the application was designed to support this journey. More detail on the implementation of each of these steps is provided in Section 3.2.

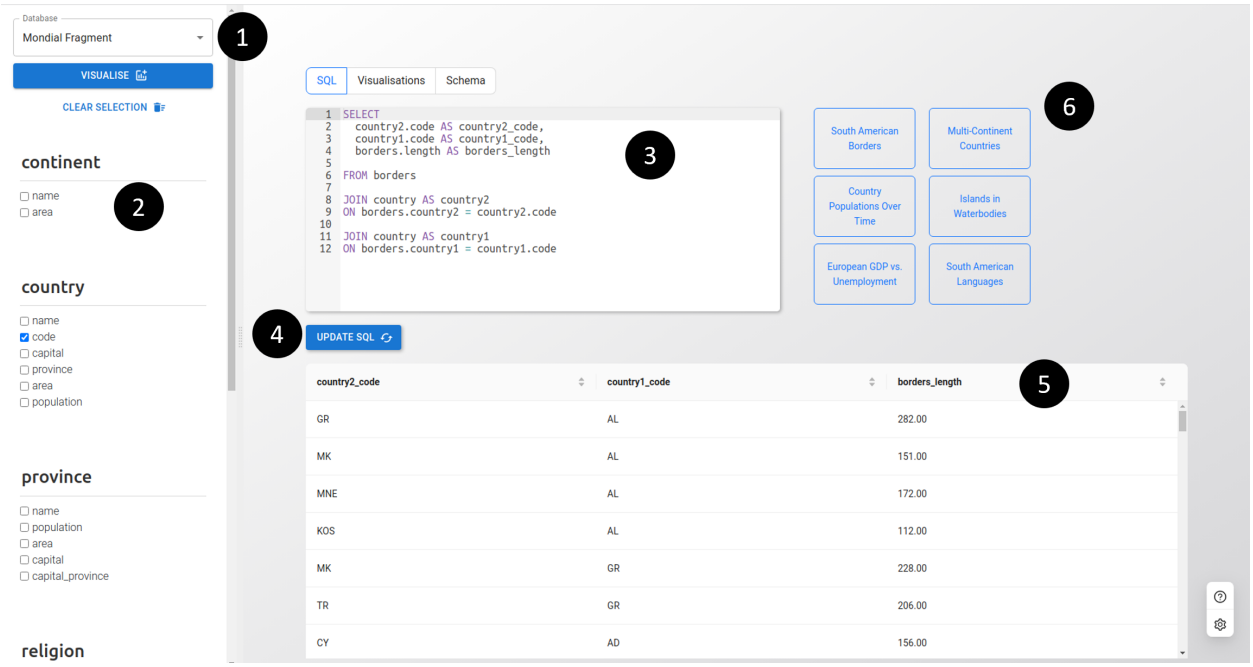


Figure 3.2: *An annotated screenshot of the application landing page*

Database Selection

The application provides a `DatabaseProfiler` class which accepts database and user details and provides a reverse-engineered `DatabaseSchema` object in JSON format, containing information about the database entities, relationships, attributes and foreign keys.

`DatabaseSchema` objects are persisted in a MongoDB collection, meaning that expensive database profiling operations need only occur once.

The example databases provided (shown in Figure 3.2, Item 1) have been profiled in this way and can be looked up by their MongoDB identifiers. This design ensures that the application can support future functionality to allow users to profile their own databases and persist this profile for future analysis.

Field Selection

The application was designed with point-and-click attribute selection as its primary interface (Figure 3.2, Item 2). The main rationale for this design — as opposed to having a SQL query as the first step of analysis — was to increase the user-base that would be able to benefit from the tool (to include those without SQL experience).

To achieve this, the `DatabaseSchema` object provides separate arrays of `Entity` and `Relationship` objects, each containing an array of `Attribute` objects. By iterating over these arrays the user interface displays all entities and their attributes, followed by all relationships and their attributes - omitting any relationships with empty attribute lists.

Each `Relationship` object also contains an array of `Edge` objects. Treating entity-relationship information as a graph data structure, the edges capture information about which nodes (i.e. entities and relationships) have connections to one another via foreign keys.

This design allows the user interface to be updated as attributes are selected, by displaying only those entities and relationships that have a connection to the current selection. The rationale for this design was to make use of underlying schema information to add clarity to the user interface and provide guidance to users.

Query Generation and Query Updates

After field selection, clicking 'Visualise' sends an Axios request to the back-end and generates all required application information – including a SQL query to generate the visualisation dataset (Figure 3.2, Item 3). The resulting dataset is presented to the user with the option to sort the returned data in order to aid understanding of the analysis dataset (Figure 3.2, Item 5).

To generate a SQL query from selected fields, `Entities` and `Relationships` in the `DatabaseSchema` object hold `ForeignKey` objects which provide the names of foreign key attributes along with the object and attributes to which they correspond.

To allow more advanced users to amend the generated SQL, the query is presented in an editor with an 'Update SQL' button (Figure 3.2, Item 4) to regenerate application information, by sending an updated request to the back-end. [1]

Illustrative examples

The application provides a number of examples to demonstrate functionality and highlight interesting features (Figure 3.2, Item 6). The examples are implemented by retrieving information persisted in MongoDB. In future this design will allow the application to support saving and retrieving user sessions.

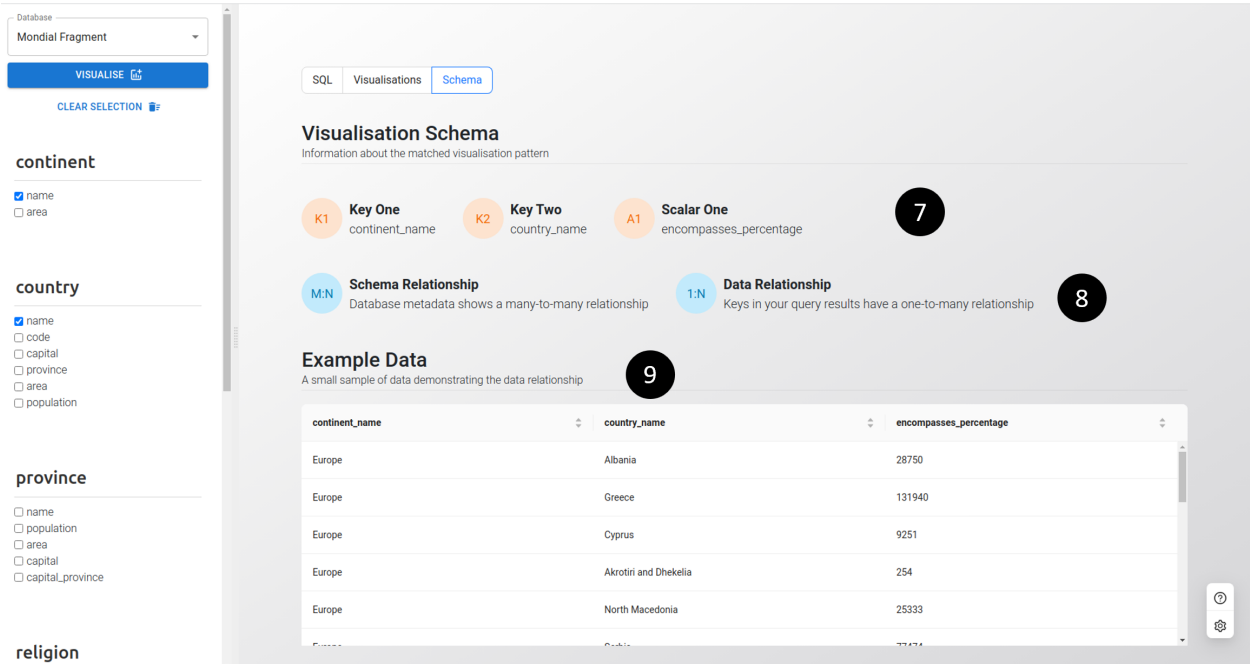


Figure 3.3: *An annotated screenshot of the application's 'Visualisation Schema' information page*

Visualisation Schema Information

McBrien and Poulovassilis [1] introduce the concept of a **visualisation schema pattern** – which are patterns formed by the combination of database entities, their participation in relationships and their attributes (including attribute types). Visualisation schema patterns can in turn be mapped to specific graphical representations that most appropriately represent the data.

This visualisation schema is mapped as a `VizSchema` object, which stores all rele-

[1] The submission of raw SQL queries from the front-end presents potential security concerns. However, the application is designed to be used by analysts on their own databases, and using their own credentials, meaning that malicious use of this functionality is unlikely.

vant attributes (see Figure 2.5) as individual JSON fields (e.g. `KeyOne, KeyTwo, ScalarOne, ScalarTwo`). To ensure the centrality of these concepts to the analytical workflow, these fields are presented in the user interface (Figure 3.3, Item 7).

The *Schema Relationship* and *Data Relationship* are also stored in the `VizSchema` object and presented in the user interface (Figure 3.3, Item 8). The *Schema Relationship* is based on the entity-relationship as defined by database metadata and summarised by the Amazing-ER profiling tool. By contrast, the *Data Relationship* is determined by examining the relationship between keys in the actual result set of the user query. It is possible for entities to have a many-many relationship in the database schema, but for the results of a query based on these entities to exhibit only a one-many relationship, for example.

This is an important distinction that is emphasised to users, as whilst a *Schema Relationship* defines a visualisation schema pattern that will *always* hold (as new data is added to the system, for example), a *Data Relationship* gives the most appropriate visualisation schema pattern for a *specific subset* of the data, which may not hold for all possible subsets of the data. Each may be more appropriate for a different use case, with a *Schema Relationship* best supporting ongoing, automated reporting – for example – and a *Data Relationship* best supporting a one-off, ad-hoc analysis of a specific result set.

To make the data relationship as clear as possible to users, a small set of rows exemplifying the data is also presented in the user interface (Figure 3.3, Item 9) to ensure that analysis proceeds with a proper understanding of the underlying structure of the data.

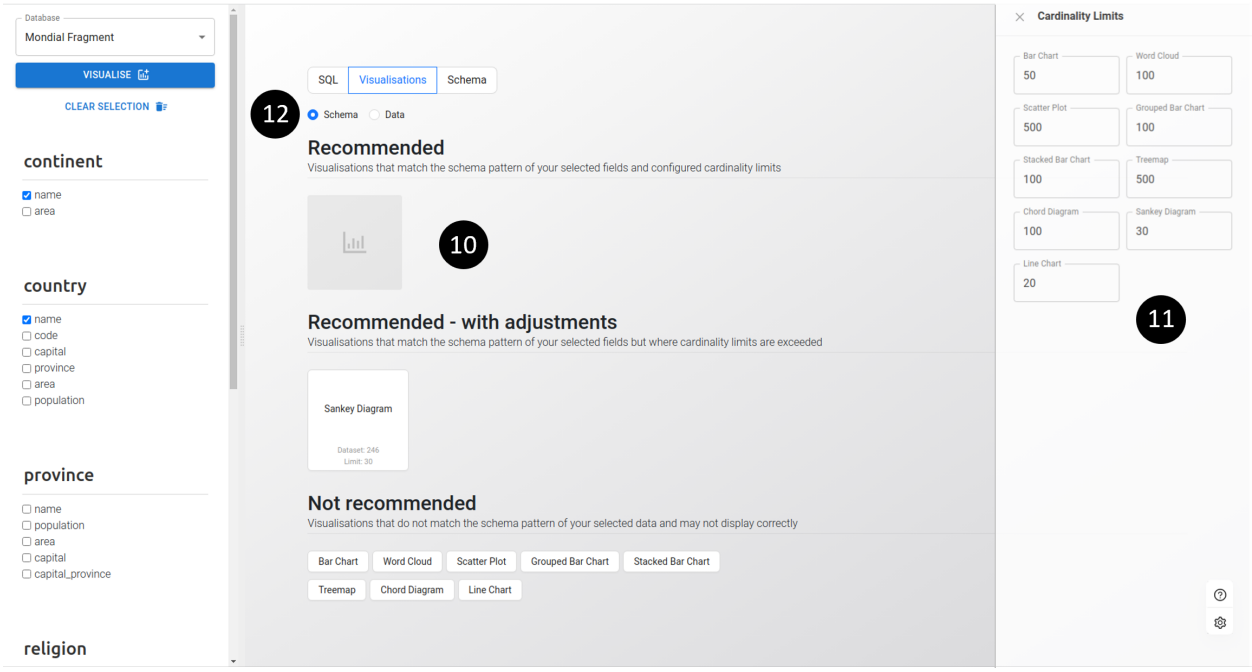


Figure 3.4: *An annotated screenshot of visualisation recommendations*

Visualisation Recommendation

Based on user-selected fields, the back-end determines an array of chart types that match the identified visualisation schema pattern, and returns this as part of the *VizSchema* object, along with numerical fields summarising the cardinalities of the selected key fields.

This array of chart types is further refined in the front-end by comparing a JSON file defining limits per chart type with the cardinalities of the keys in the returned dataset. These limits are predefined, but can be adjusted by users according to their preference using the settings menu (Figure 3.4, Item 11).

The combination of matched chart types and comparison to cardinality limits produces three lists presenting visualisations that are 'Recommended' (i.e. matching the visualisation schema and within cardinality limits), 'Not Recommended' (i.e. not matching the visualisation schema), and 'Recommended with adjustments' (i.e. matching the visualisation schema but where cardinality limits are exceeded). These distinctions are shown in Figure 3.4, Item 10.

The *VizSchema* object holds two arrays describing the chart types matching the visualisation schema pattern according to *'Schema Relationship'* (as described in the previous section) and the *'Data Relationship'*. If these relationships differ, the user is presented with an option to switch between recommendations based on each

relationship type (Figure 3.4, Item 12).

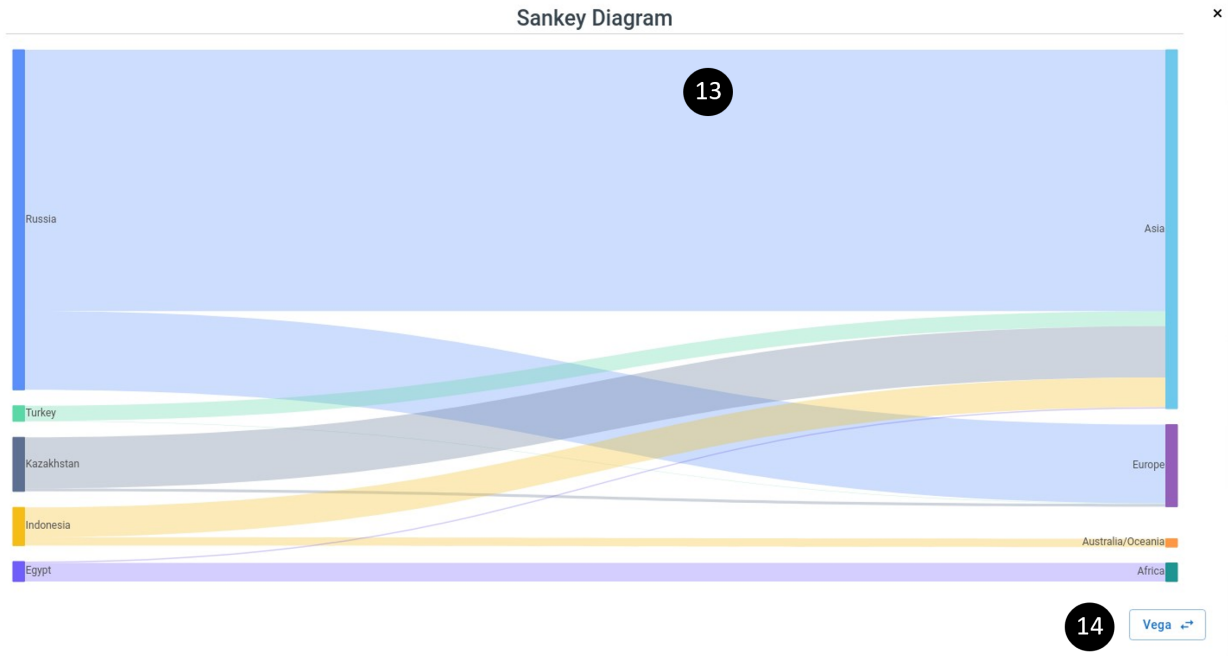


Figure 3.5: *An annotated screenshot of a Sankey Diagram visualisation built with Ant-V. The visualisation shows countries that span more than one continent, according to the Mondial database.*

Data Visualisation

Clicking on any item from the visualisation recommendations page will render a chart, using the dataset stored in the `VizSchema` object and assigning fields to elements of the visualisation using the categorisation of fields within the `VizSchema` object (e.g. `KeyOne, KeyTwo` etc.).

The application primarily uses Ant-V for chart rendering – owing to its extensive support for a range of chart types, including specification through visualisation grammar, as well as its out-of-the-box support for responsive and interactive data visualisation in a web context.[2] Figure 3.5, Item 13 shows a sankey diagram built and rendered using Ant-V.

The application also supports data visualisation using the Vega ecosystem. As part of the project a novel library was developed to programmatically produce Vega visualisations in Java, meaning that the application supports Vega visualisation through its Java back-end. In the context of this time-limited project it was found that Vega's lack of native support for responsive visualisation made it untenable

[2]Also see Section 2.1.3 for further information on the rationale behind this choice

as the primary visualisation engine for the project. However, certain visualisations have been constructed using this methodology and are offered as a proof of concept (Figure 3.5, Item 14).

3.1.5 NoSQL Database Assessment

As shown in Figure 3.1, a NoSQL application database was chosen to support this application.

This decision was driven in part by the fact that many elements of the application rely on JSON data, including core elements of the application such as the JSON format for Vega visualisation specifications and the JSON format expected for datasets in the majority of open source visualisation tools. For this reason, an application database built around the concept of native JSON documents was considered the optimal design choice.

Similarly, the nature of Vega specifications is that there are a very large number of optional fields that may be included in a given specification, and these fields and sub-objects can be nested in different ways depending on the use case. Trying to represent these relationships in a relational model would be complex, whilst also being inefficient given that very wide tables with a lot of NULL values would be required. By contrast, in a NoSQL environment, unused fields can be omitted entirely from documents - leading to a more efficient storage of the data.

Finally, MongoDB was specifically selected as the NoSQL database provider given its good documentation, broad community and close integration with Java Spring Boot (using the Spring Data MongoDB integration) [33].

3.2 Application Implementation

3.2.1 Database Schema Mapping

Existing Imperial research and tooling

A guiding principle of this project was to leverage and advance existing research at Imperial College London where possible. To that end, the project made use of the Amazing-ER tool for database schema profiling. Amazing-ER was developed by students at Imperial College London, providing a Java environment for database schema creation, manipulation and analysis [34, 35]. The key Amazing-ER functionality used in this project is the ability to connect to a database and reverse engineer it to produce a set of Java classes describing its entities, relationships and attributes. To ensure a clear separation of concerns, wrapper classes were created to house and extend this external functionality – ensuring that the project is flexible enough to use alternative tools in the future if necessary.

Amazing-ER database profiling

The `DatabaseProfiler` class created as part of this project uses the Amazing-ER library as a starting point for database profiling.

As shown in Figure 3.6 the `DatabaseProfiler` makes a call to Amazing-ER's `ER.initialize()` to generate an in-memory H2 database to be used for ingestion and analysis of the source database tables. The call to `relationSchemasToERModel(...)` analyses the provided database and returns a `Schema` object, which contains the classes shown in Figure 3.7.

```
1 ER.initialize();
2 Reverse reverse = new Reverse();
3 amazingErSchema = reverse.relationSchemasToERModel(
4     databaseType, host, port, databaseName, user, pw);
5 profileEntityForeignKeys(connectionString, user, pw);
6 schema = new DatabaseSchema(
7     amazingErSchema, connectionString, entityForeignKeys)
```

Figure 3.6: *Simplified logic from the DatabaseProfiler class, showing how Amazing-ER methods are used to construct a DatabaseSchema object.*

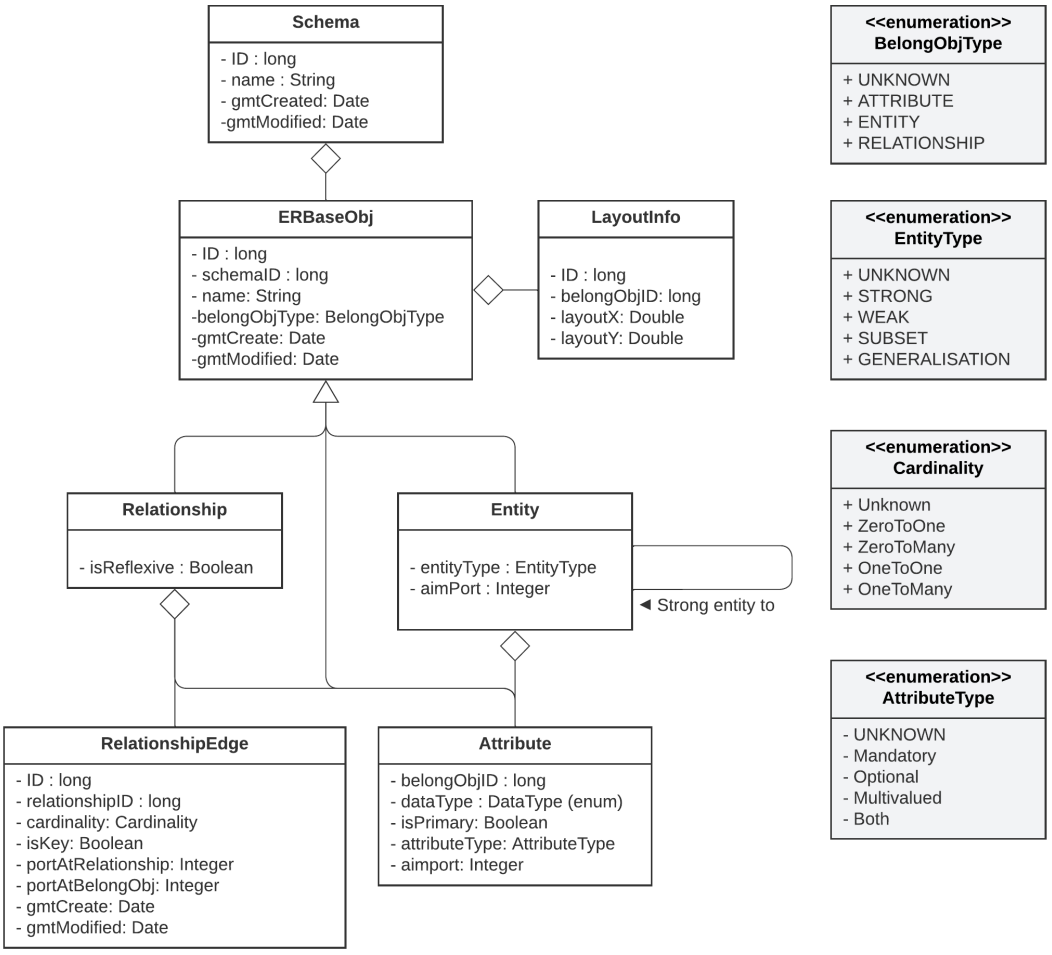


Figure 3.7: *A UML diagram showing the classes from the Amazing-ER library that were built upon as part of this project.*

Customising and extending the Amazing-ER tool

The Amazing-ER tool is comprehensive, totalling more than 60 classes and interfaces, with extensive functionality including database creation and data-definition language generation. Given that much of this extended functionality is not necessary for this project, wrapper classes were created, following the *Decorator* and *Adapter* design patterns [36], in order to simplify the classes generated by Amazing-ER.

In addition to reducing complexity, the wrapper classes allow the addition of functionality to support the goals of this project, as well as ensuring that there is a separation of concerns — supporting the substitution of other database profiling tools in the future, if required.

The original Amazing-ER class diagram is shown in Figure 3.7, whilst the classes

created as part of this project are shown in Figure 3.8. The `DatabaseProfiler` class takes an Amazing-ER schema at construction, and adapts the contained classes as required.

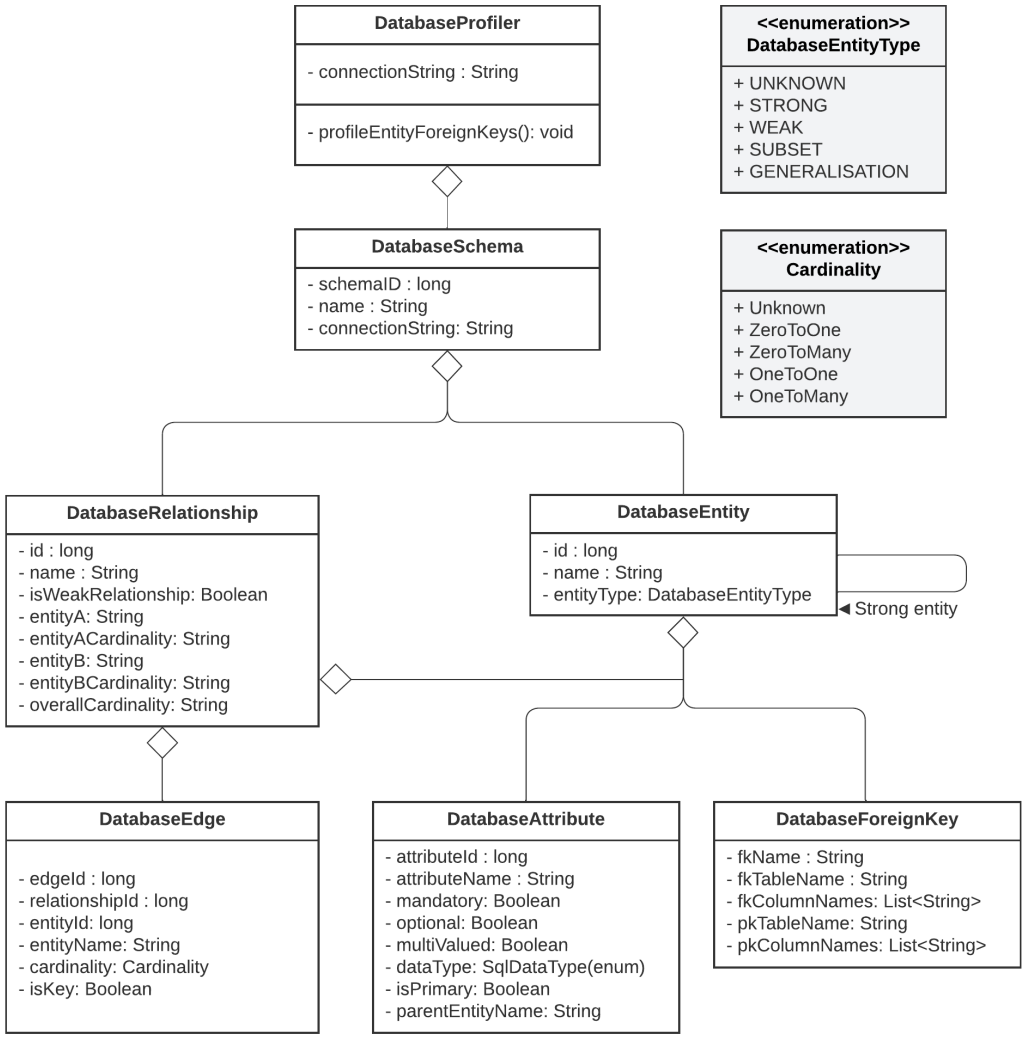


Figure 3.8: *A UML diagram showing database profiling classes used as part of this project. These classes are wrappers and extensions of the Amazing-ER library.*

The key changes made to the Amazing-ER object-orientated design were as follows:

Simplifications of the Amazing-ER design

- The full Amazing-ER library is imported as an external .jar file while only the wrapper classes are defined as part of this project. This is much more manageable for future maintenance, with only 9 wrapper classes exposed –

compared to 63 classes/interfaces as part of Amazing-ER.

- Inheritance from ERBaseObj is removed, as the required attributes from Entity, Relationship and Attribute have minimal overlap, and inheritance incurs a cost in terms of complexity and coupling between classes.
- Extraneous attributes are removed, particularly metadata (such as creation time) and information related to unused Amazing-ER functionality (e.g. layout and data-definition language generation).
- Some enum fields have been replaced. For example, the `AttributeType` enum class is replaced with boolean flags (to simplify interpretation of the 'both' value — where attributes can be optional and multi-valued, for example).

Additions and extensions to the Amazing-ER design

- A `ForeignKey` class has been added, along with a method to generate this information from JDBC. Foreign key information is calculated at the point of database profiling and stored against entities and relationships which hold this information. The additional information is used for the automated generation of SQL queries to create visualisation datasets.
- Information about connections between `Entity` and `Relationship` objects has been promoted to the `DatabaseRelationship` class. This logic essentially traverses the relationship edges, treated as an adjacency list, and persists the information so that it can be traversed once during database profiling, rather than at runtime.

  This logic also resolves the information to a higher level, meaning that where Amazing-ER gives the cardinalities at the 'edge' level of detail, this additional logic summarises it also at the 'relationship' level. So, for example, a connection between entities many be '0..1' at one end and '1..N' at the other end. Summarised at the relationship level this is a one-to-many relationship.

3.2.2 Schema-aware Field Selection

In keeping with the schema-driven ethos of this application, field selection is guided by underlying schema information. As a user selects fields, the displayed list of entities and relationships is updated to reflect only those that have a relationship to something that has already been selected. This is shown in Figure 3.9

This functionality caters to one-to-many relationships, weak entity relationships, many-to-many relationships (including both the entities and relationships involved) and 'is-a', subset relationships.

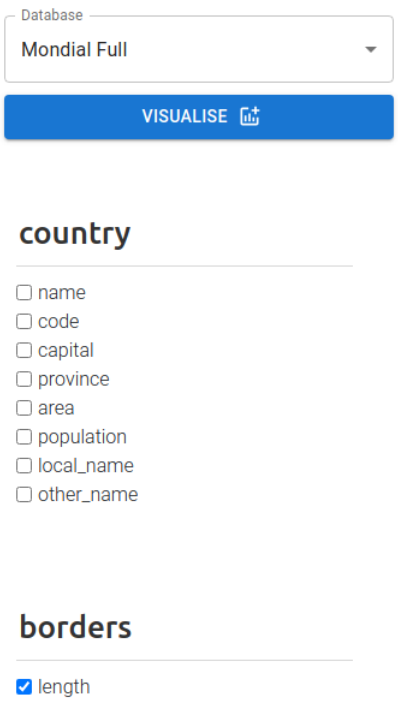


Figure 3.9: *The application UI during field selection. Selecting a field on the 'borders' relation in the Mondial database limits selection to the 'Country' entity, with which it has a reflexive relationship*

.

This is achieved in the front-end, via the following logic contained in the `EntityList` and `EntityComponent` React components:

- Checking or un-checking an attribute in the user interface triggers an update to a set tracking currently selected Attributes.
- The same action also triggers an update to sets of Entities and Relationships containing currently checked attributes. These are referred to as 'active' entities and relationships.

- Two further sets are then updated, capturing `reachableEntities` and `reachableRelationships`. These are populated by first filtering the list of schema relationships to those that contain an 'active' entity or relationship. 'Reachable' entities and relationships can then be calculated as any of the objects listed in the adjacency list information -– which is summarised on the Relationship object.
- For subsets, additional logic is required to identify the relationship in both directions, by using the `BelongStrongEntity` attribute of the parent entity.

Optimising field selection

Given that this logic is triggered with every check and un-check event by a user, it was important to ensure an efficient implementation. This was achieved as follows:

- The sets of entities and relationships containing checked attributes are populated by querying the set of currently checked attributes and extracting the `parentEntityName` (which was an added as an additional field during extensions to Amazing-ER). Doing so ensures that only a small list of checked attributes needs to be iterated through, as opposed to traversing the full set of entities and relationships on each user interaction, which would have linear complexity with respect to the number of entities contained in the database.
- Sets are used to store 'active' attribute, entity and relationship information to increase the efficiency of lookups -– achieving an amortised O(1) time complexity, compared to O(N) complexity using arrays [37].

Field selection payload

The sets of checked attributes and reachable entities and relationships are maintained as the user interacts with the list of fields.

Once the selection has been finalised and submitted, this also means that a reduced `Schema` object can be sent to the back-end, in order to reduce the size of the payload being sent across the network, which could cause latency for large or complex database schemas.

Only information about 'active' entities and relationships is sent in the back-end payload whilst attribute lists for these entities and relationships are reduced to those that have been selected. Similarly, foreign key information is only sent for those foreign keys that relate to 'active' entities and relationships.

The resulting reduced payload is demonstrated in Figure 3.10.

Figure 3.10: *API endpoints and payloads during communication between the application client and server.*

3.2.3 Query Generation

Given the project's aim to provide an no-code interface for less experienced users, a key piece of functionality was the automated generation of SQL queries based on user field selection. SQL query generation entails a number of challenges which are outlined here, followed by a brief overview of the logic followed for query generation in each relationship type. The goals for SQL query generation were as follows:

- SQL query generation logic must locate the correct foreign key and attribute information – which in some cases is stored on the entity object, but in some cases – such as many-many relationships – is stored on the relationship object.
- Query generation must also be capable of building joins that involve composite keys involving an arbitrary number of fields.
- Queries must avoid collisions in field names between entities by aliasing attributes appropriately. Similarly, in the case of self-joins – such as those used in reflexive relationships – there must be a mechanism to provide multiple different aliases for the same entity.

The logic used to achieve these goals is contained in the back-end `VizSchemaMapper` class, and is summarised at a high level as follows:

Basic entities:

- Access the entity's list of attributes as a stream
- Map each attribute to create a string of the format *entityName.attributeName AS entityName_attributeName*
- Collect the generated strings, delimited by a comma as the 'SELECT' statement and add it to the entity's name as the 'FROM' clause.

One-to-many, weak entity and subset relationships:

- Access foreign key field names and their corresponding primary keys on whichever entity object holds foreign key information. [3]
- Assign the entity holding foreign key information to the 'FROM' clause, assign the other entity to the 'JOIN' clause

[3] Foreign key and primary key information are stored as ordered arrays to account for composite keys

- Map each field in the foreign key and primary key arrays to the format *entityName.attributeName*. This is particularly important to avoid field name collisions between tables.
- Map across both arrays to produce the format *entity1Name.attributeName = entity2Name.attributeName*
- Collect the resulting list into an 'ON' clause string, with an 'AND' delimeter to chain join conditions if needed – e.g. *attributeA = attributeA AND attributeB = attributeB*
- Generate an aliased 'SELECT' list in the same way as for basic entities, but with a loop over both entities.

Many-to-many (non-reflexive) relationships:

- Access foreign key field names and their corresponding primary keys on the relationship object. Based on earlier filtering, only one relationship object will be in the schema (which will be the relationship object containing user-selected attributes).
- Assign the relationship name to the 'FROM' clause
- Use the size of the foreign keys list to determine the number of 'JOIN' clauses needed. Add the specified number of joins, at the same time creating aliased foreign/primary key pairs to be built into an 'ON' clause string (using the same method as one-many joins).

Reflexive many-to-many relationships:

- In reflexive many-many relationships the same logic is followed, with the exception that field aliases are created from foreign key field names in order to provide distinct aliases for the entity.

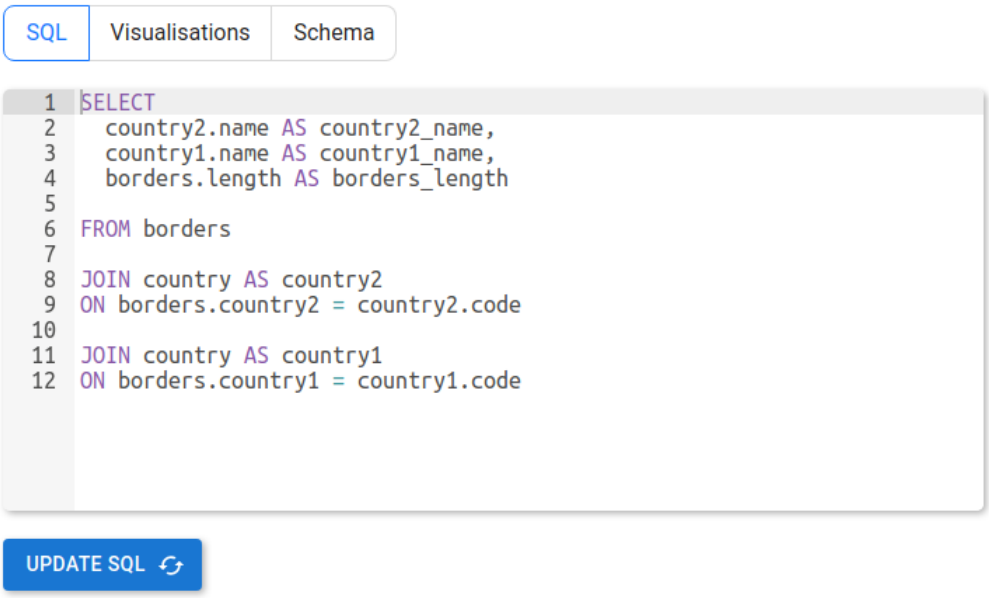


Figure 3.11: *An application screenshot showing the SQL code generated by the selection of the reflexive many-to-many relationship between* country.name *and* borders.length *in the Mondial database*

3.2.4 Visualisation Schema Mapping

The `VizSchemaMapper` class is also responsible for determining the visualisation schema pattern from a provided `DatabaseSchema` object and returning a `VizSchema` object to be visualised by the front-end.

The `VizSchemaMapper` class provides a public `generateVizSchema` method which can be used to create a `VizSchema` object. The `generateVizSchema` method first uses the information provided by the DatabaseSchema object on the attributes, entities and relationships that have been selected in the front-end to determine the relevant visualisation schema pattern.

A number of private methods are then called depending on the type of relationship identified - e.g. `generateBasicEntitySchema`, `generateWeakEntitySchema` etc. The role of these methods is to identify the role that selected fields should play in the visualisation schema (e.g. 'Key One', 'Key Two' and 'Scalar One' etc.) based on their characteristics such as their data type and any constraints defined in the database schema.

For example, any field which is identified as a primary key in the `DatabaseSchema` object can be assigned to a 'key' field in the visualisation schema pattern. In ad-

dition to the patterns outlined in *Towards Data Visualisation based on Conceptual Modelling* [1], if no primary key fields are present an additional SQL query is run to check whether the field is unique – and if so treats this as a potential visualisation 'key' field. An example of this pattern is the *country* table from the Mondial database - in which the primary key is *code*, but in which the *name* field is also unique, and presents a more informative field for visualisation.[4]

Identifying data relationships and cardinalities

The `VizSchema` class is also responsible for sending additional queries to identify the relationship exhibited by the actual subset of data returned by a user query, in order to provide sample datasets to be displayed in the application, as well as to identify whether the selected data exhibits a relationship that differs from the relationship determined by the database schema.

Given that the `VizSchema` class already holds information on the relevant fields, their role in the visualisation schema pattern, and the SQL query defined by the application/user – this information can be used to construct these additional queries, as shown in Figure 3.13. This query defines a temporary table, to allow a number of `VizSchema` fields to be retrieved in the construction of the `VizSchema` object, without repeated database queries.

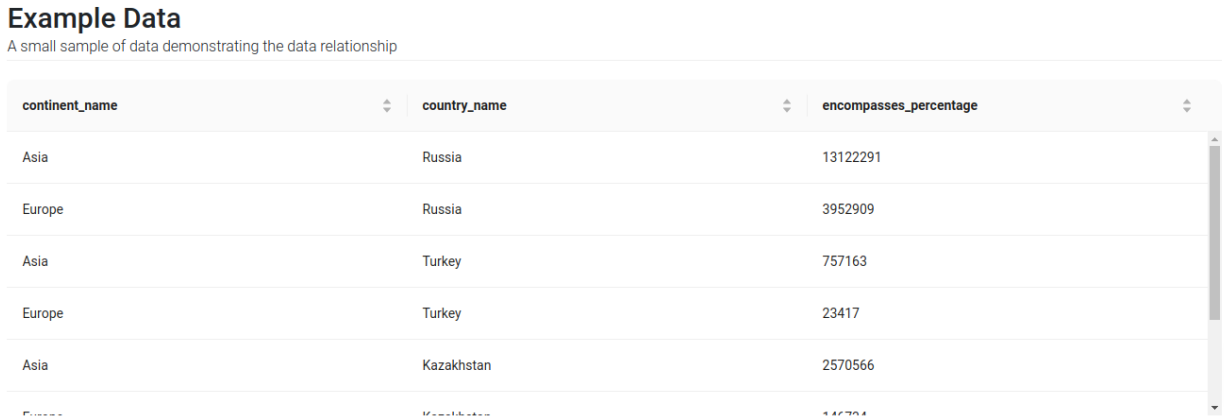


Figure 3.12: *An application screenshot showing the example dataset presented to users for a many-many relationship. The example shown is the relationship between* 'country' *and* 'continent' *in the Mondial database - an example in which a many-many relationship may not be obvious to the user.*

[4] As an additional check - to be a candidate for a 'key' field the datatype of the field must also be non-numeric, as it is common for numeric fields to have unique values, but these fields should not play the role of keys in effective visualisations.

```
CREATE TEMPORARY  TABLE temp_cardinality_data AS
  WITH raw_data AS (
            /*--Frontend user SQL Query--*/
        )

  ,a_to_b AS (
  SELECT keyOneAlias, count(keyTwoAlias) AS a_count;
  FROM raw_data;
  GROUP BY keyOneAlias);

  ,b_to_a AS (
  SELECT keyTwoAlias, count(keyOneAlias) AS b_count);
  FROM raw_data);
  GROUP BY keyTwoAlias);

  SELECT raw_data.*, a_to_b.a_count, b_to_a.b_count
  FROM raw_data;
  JOIN a_to_b ON a_to_b.keyOneAlias = raw_data.keyOneAlias
  JOIN b_to_a ON b_to_a.keyTwoAlias = raw_data.keyTwoAlias);
    );

-- The relationship exhibited by the selected dataset can then be
-- determined by finding the maximum for both counts
-- If max_a and max_b are > 1, for example, the relationship is many-many
SELECT
  max(a_count) as max_a,
  max(b_count) as max_b
FROM temp_cardinality_data;

-- An example dataset can then be constructed as follows.
-- This dataset would show records exhibiting a many-many relationship
SELECT * FROM temp_cardinality_data
WHERE a_count > 1 AND b_count > 1;
```

Figure 3.13: *A summary of the SQL query used by the VizSchema class to determine the relationship between keys in a given dataset*

The presentation of the resulting information in the application interface is shown in Figure 3.12.

3.2.5 Visualisation Construction and Rendering

The application provides two mechanisms for constructing and rendering visualisations to the screen, either via Ant-V or Vega, as these were the leading open-source utilities assessed (see Section 2.1.3). The primary visualisation method used throughout the project is Ant-V, given its more advanced support for responsive, web-based visualisation.

Considerable work was undertaken as part of the project to contribute a programmatic method for specification of Vega visualisations in Java (see section 4.4). However, given the need for further work to improve the responsiveness of Vega visualisation in a web context, Vega visualisations are only supported for a smaller subset of charts in this project, with these visualisations only in the beta testing phase.

Data visualisation using Ant-V

The Ant Group maintains Ant-V, an ecosystem of charting libraries built on top of a bespoke visualisation engine [38]. The provided libraries are tailored to network visualisation [39], geo-spatial visualisation [40], and visualisations optimised for mobile technology [41], as well as the G2 visualisation grammar for custom visualisation [42].

Ant Design Charts is a library built on top of this ecosystem to provide a simple interface for constructing common chart types in the context of React web applications [43]. This high-level library was sufficient for all visualisation construction and rendering required by the project to date.

Ant Design Charts accepts inputs through a configuration interface, and then constructs charts based on this specification using the underlying G2 grammar.

The configuration interface is a JavaScript object that accepts arguments including:

- the dataset for visualisation
- the fields that map to chart dimensions and channels
- specification and formatting of axes, legends and labels
- interactive chart components such as brush-filtering and tooltips

This specification is flexible, allowing for extensive customization including through bespoke JavaScript, HTML and CSS – but also provides comprehensive default behaviours, meaning that many configurations can be very concise. Figure 3.14, for example, shows a minimal chord diagram configuration in Ant Design Charts.

```
const data = vizSchema.dataset.map((item: any) => ({
    source: item[vizSchema.keyOneAlias],
    target: item[vizSchema.keyTwoAlias],
    value: parseFloat(item[vizSchema.scalarOneAlias]),
  }));

  const config = {
    data: data,
    sourceField: "source",
    targetField: "target",
    weightField: "value",
  };
  return <Chord {...config} />;
```

Figure 3.14: *React code used to define a chord diagram using the Ant-V* 'config' *object*

Data visualisation using Vega

The majority of the effort required to display data using Vega visualisation is in the construction of a JSON visualisation specification. The specification acts as a declarative interface, providing a visualisation grammar which can be interpreted and rendered by a set of tools developed and maintained as part of the Vega ecosystem.

In this project, the construction of Vega specifications is handled in the back-end, using the *jVega* library developed during this project. This is described in detail in Section 4.4.

The key implementation detail here, which is facilitated by the design of *jVega*, is that Vega visualisation template specifications are stored in MongoDB. Once loaded in response to a front-end request, the object-orientated design allows the specification to be quickly customised to a user specification by inserting a custom dataset and transformations.[5]

Once constructed, the react-vega library can be used to render the visualisation, simply by passing a Vega specification. For example, with a JSON object representing the desired specification stored in the variable *vegaSpec*, the visualisation can be constructed and rendered in react-vega using `<Vega spec={vegaSpec}/>`.

[5]The majority of Vega templates were taken from examples provided through https://vega.github.io/editor. However, the Sankey diagram template is based on developments by other community members. See, https://github.com/PBI-David/Deneb-Showcase/tree/main

Optimising visualisations

In order to support the visualisation of larger datasets, as far as possible a single copy of the visualisation dataset is stored as a variable in the front-end and passed by reference to different visualisations types. This design helps to ensure both time and space efficiency for the application, by separating the visualisation dataset from the visualisation specification.

However, given limited control over the internal implementation of visualisation rendering tools, the efficiency of these processes cannot be guaranteed. For example, some Vega transformations create copies of the dataset internally.

This is an area where further development of solutions built around Vega may be beneficial, as this approach provides complete control over specification, as well as providing opportunities to move computationally intensive elements of the workload to the back-end.

jVega: A Java Binding for the Vega Visualisation Grammar

4.1 Problem Statement

Vega was identified as one of the leading visualisation ecosystems for use in this project, and was the primary focus in the early stages of project development. The rationale for this selection is outlined in more detail in Section 2.1.3.

Java was selected as the optimal language for back-end development in the project, given its mature ecosystem of libraries and APIs for database interaction and data-intensive applications. The project was also designed to extend existing Imperial College London research projects which were built in Java.

Given these design decisions, a clear gap was identified in the absence of a Java binding for Vega visualisation specification. Whilst JavaScript APIs exist for the creation of Vega specifications, an initial design decision was made to create a clear separation of concerns – with the core application logic being specified in the Java back-end.

jVega was developed as part of this project in order to fill this gap. The short-term goal of jVega was to support Vega visualisation as part of this application, but the developments also form a contribution to the wider data visualisation community.

4.2 Project Contribution

Vega's creators emphasise the rationale behind its declarative JSON interface as being the provision of a "means for writing *programs* that generate visualizations", with Vega abstracting away the construction and rendering of visualisations themselves [44].

Central to the Vega project is therefore the collection of APIs and libraries that allow interaction with the Vega grammar using a range of programming languages. So far the community has developed support for a large number of languages – including Python, Scala, R and JavaScript, but Java is notably absent from the available programming language bindings [45].

The absence of a Java interface for Vega means that its accessibility is currently limited for a large community of engineers and researchers whose language of choice is Java, as well as for enterprises for which Java is central to their data infrastructure.

4.3 jVega Design

4.3.1 Design Requirements

The key points in the jVega design specification were as follows:

- Provide a fluent Java API to promote clear, readable code.
- Follow an object-orientated design, allowing the construction of Vega specifications as a composition of hierarchical objects which can be varied programmatically - in keeping with the approach outlined in the *Grammar of Graphics* [8].

  Other libraries for Vega bindings have preferred an approach that constructs Vega specifications through string manipulation or direct manipulation of JSON objects. This approach was discounted as the absence of object-orientated design principles can create a code base that is more difficult to extend and maintain.
- Leverage Java's static typing to make Vega's specification mechanics more transparent to developers (e.g. surfacing valid object and attribute nesting through auto completion).

- Use appropriate design patterns to abstract away complexities in Vega specifications (for example by providing a flatter interface for deep nesting of components).
- Minimise the possibility of creating invalid Vega specifications through Java type checking.
- Provide mechanisms to serialize and deserialize Vega specifications between a Java class hierarchy and a JSON string representation.

4.3.2 High-Level Design

A Vega specification is a JSON object, which may include sub-objects to specify the characteristics of *Scales, Axes, Legends and Marks*, as well as the *Data* underlying the visualisations (and any associated transformations) and any *Signals* (i.e. variables and user inputs) that interact with the data.

At a high level, jVega represents each of these components as Java class, along with a builder class (`BuildSpec`) to surface methods for constructing the specification. To achieve a comprehensive Vega specification, many further classes are provided - which are discussed in more detail in the implementation section below.

The UML diagram in Figure 4.1 shows the relationship between the parent Vega specification and the top-level classes of which it is composed.

Composition is used to describe the relationship between a specification and its sub-components, in order to capture the fact that specification components hold no meaning outside the specification itself. However sub-components may well be defined separately to a specification, to allow sharing and re-use of components between specifications, as well as to create cleaner and more readable code.

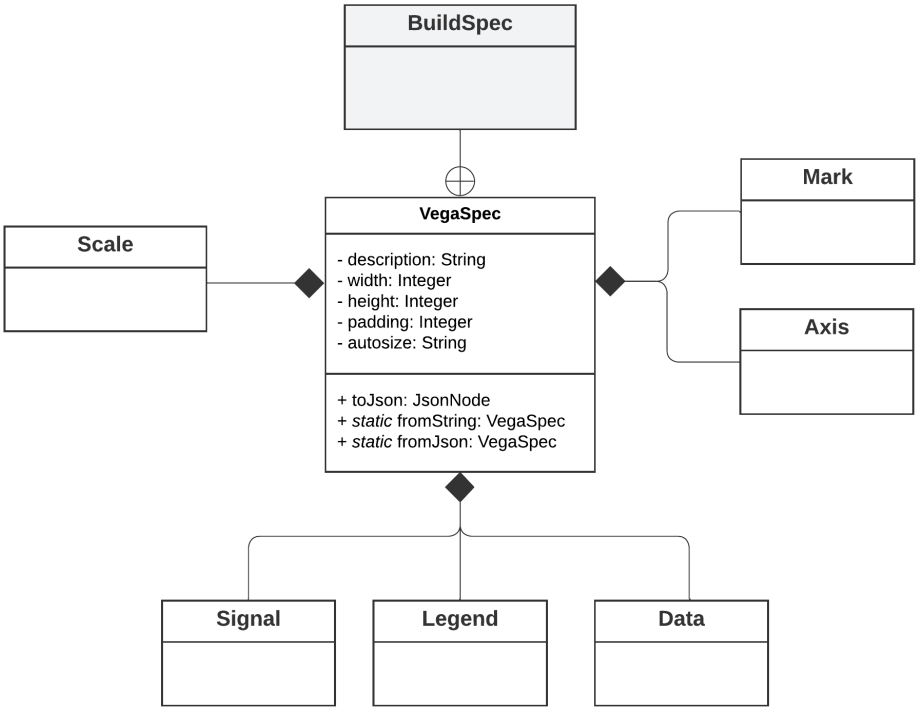


Figure 4.1: *A simplified UML diagram showing a high-level overview of the jVega object-orientated design. Fields and methods have only been included for the* `VegaSpec` *class for clarity. Details of the exact specification of sub-components is shown in Section 4.4*

4.3.3 Design Patterns

In order to achieve a fluent, intuitive interface for building Vega specifications, the project makes use of creational object-orientated design patterns. In particular two design patterns are used, according to their specification in *Design Patterns: Elements of Reuseable Object-Orientated Software* [36].

Builder pattern

The Builder Pattern is designed to allow the construction of complex objects by providing a separate builder class which exposes methods for the inclusion of each class attribute and allows the developer complete control over how an object is constructed [36]. In the context of Vega specifications this is particularly important given that there are a very large number of possible attributes, and the object representation will vary greatly from specification to specification.

Factory pattern

The Factory Pattern abstracts the creation of a complex object and encapsulates it in a single method. These factory methods are responsible for constructing the object, meaning that for objects with a potentially large number of configurations, factory methods can provide clarity to developers through the parameters that are expected. In the context of jVega - this is important in providing a simple interface to developers, abstracting any complex nesting of attributes and signposting valid attribute combinations through factory methods.

4.4 jVega Implementation

4.4.1 Vega Scales

Vega scales have a large number of shared characteristics, such as a name, a scale type and the dataset, fields and range which define their representation.

However, scales can also be specialised - for example as a linear, logarithmic, ordinal or banded scales [46].

Given these characteristics the jVega object-orientated design implements this via inheritance – allowing all scales to share characteristics, while allowing specialisations of scales to introduce additional attributes and behaviours. This design is shown at a high level in Figure 4.2 and in detail in Figure 7.1.

Overview

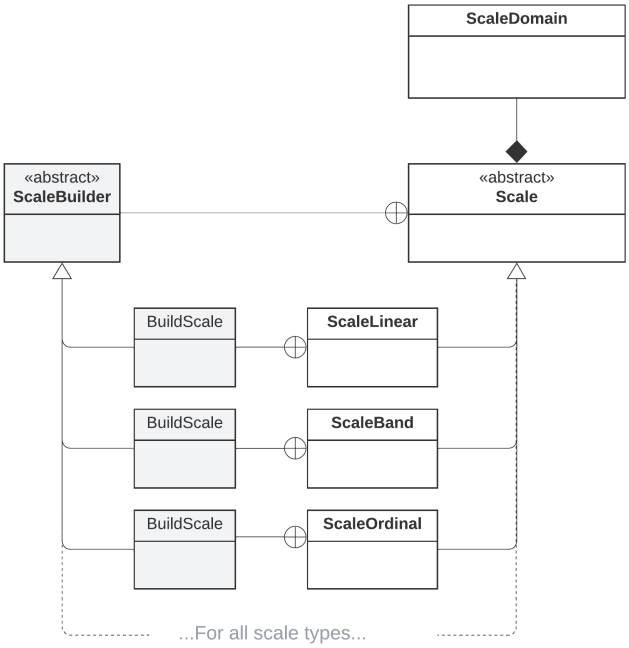


Figure 4.2: *A simplified UML diagram showing jVega's object-orientated design for Vega Scales. Specialisations of scales inherit from an abstract* `Scale` *parent, with each class including a inner builder class.*

The use of the builder pattern along with inheritance allows for the construction of a Vega scale specification with a fluent interface as well as clear type checking to more readily produce valid Vega specifications. The jVega code and resulting specification (after object deserialisation) are shown in Figures 4.3 and 4.4.

jVega scale implementation and outputs

```
1  VegaSpec scaleExample = new VegaSpec.BuildSpec()
2      .setDescription("Scale Example")
3      .setNewScale(new BandScale.BuildScale()
4          .withName("xscale")
5          .withDomain(ScaleDomain.simpleDomain("rawData", "scaleFieldName"))
6          .withRange("width")
7          .withPadding(0.05)
8          .build())
9      .createVegaSpec();
10
11 scaleExample.toJson().toPrettyString();
```

Figure 4.3: *A simple scale outlined with jVega and serialized using the .toJson() method*

```
{
  "description": "Scale Example",
  "scales": [
    {
      "name": "xscale",
      "domain": {
        "data": "rawData",
        "field": "scaleFieldName"
      },
      "range": "width",
      "round": true,
      "type": "band",
      "padding": 0.05
    }
  ]
}
```

Figure 4.4: *The resulting specification produced from the code in Figure 4.3*

4.4.2 Vega Datasets

Vega provides the ability to define mutliple 'data' objects, each containing a separate set of data values, within a single visualisation specification. This is particularly important for defining complex visualisations that visualise data at different levels of aggregation on the same canvas.

Significantly, a single source dataset can be passed to a specification, with further Vega data objects dynamically created based on this input, which allows the standardisation of the expected shape of data input across all specifications.

Central to this dynamic dataset creation are the `Transform` objects which are defined within each data object. Transforms act on a source dataset to produce new fields or change the shape and aggregation of the dataset. Transforms vary from simple formulas (based on Vega's own custom expression language), to spatial, hierarchical, path, and cross-filtering transformations [47].

Overview

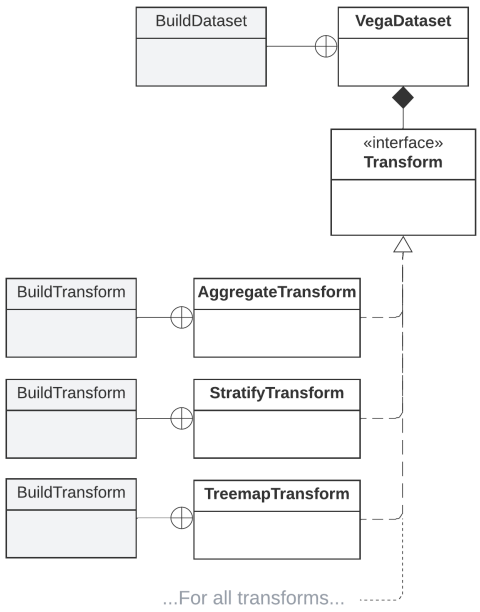


Figure 4.5: *A UML diagram showing jVega's object-orientated design for Vega Data objects. The Data object expects a list of objects which implement the Transform interface.*

Vega provides a number of ways to specify the source data for visualisation, namely through file URLs, JSON value objects or references to other Vega datasets. When providing data directly as JSON, data must be passed as a list of objects, with an object per row and a key-value pair for each attribute in the row.

To support this, jVega provides a number of factory methods to simplify data ingestion from different sources, as well as a utility class, 'JsonData' to assist in translating data (including JDBC result sets) into a valid format for Vega specifications. Examples of these methods are shown in Figure 4.6.

jVega Dataset: implementation and outputs

Figure 4.6 demonstrates the creation of three example datasets. The dataset *urlExample* loads data via reference to a URL, meaning that only the URL itself is defined in the specification. The *jsonExample* dataset loads a JSON object into the specification itself from a local file.[1] The final dataset, *derivedExample*, creates a dataset with an existing Vega dataset as its source. The use of a variable, *sharedData*, to define this common dataset name is an example of how a Java language binding can

[1]The barData.json file used in the example is an example containing only three records as displayed in Figure 4.7

contribute robustness to the creation of Vega specifications.

```
JsonNode jsonData = JsonData.readJsonFileToJsonNode("barData.json");
String dataUrl = "data/cars.json";
String sharedData = "urlExample";

VegaSpec dataExample = new VegaSpec.BuildSpec()
  .setDescription("Data example")
  .setNewDataset(VegaDataset.urlDataset(sharedData, dataUrl))
  .setNewDataset(VegaDataset.jsonDataset("jsonExample", jsonData))
  .setNewDataset(VegaDataset.sourceDataset("derivedExample", sharedData))
  .createVegaSpec();
```

Figure 4.6: *Examples of the jVega factory methods supplied for dataset creation as well as the JsonData utility class for preparation of valid Vega value sets.*

The resulting specification produced by the code in Figure 4.6 is shown in Figure 4.7.

```
{
  "description" : "Data example",
  "data" : [ {
    "name" : "urlExample",
    "url" : "data/cars.json"
  }, {
    "name" : "jsonExample",
    "values" : [ {
      "category" : "A",
      "amount" : 28
    }, {
      "category" : "B",
      "amount" : 55
    }, {
      "category" : "C",
      "amount" : 43
    } ]
  }, {
    "name" : "derivedExample",
    "source" : [ "urlExample" ]
  } ]
}
```

Figure 4.7: *The resulting specification produced from the code in Figure 4.6, after serialization with jVega's toJson() method. Here the* jsonExample *dataset was loaded from a local JSON file (*barData.json*) containing only three records for example purposes.*

jVega Transforms: implementation and outputs

Similar to jVega `Scale` objects, `Transform` objects are implemented as separate classes, with individual builder classes. This is to provide modularity, which will help to ensure simple navigation, maintenance and extension of the library for developers.

Unlike scales, however, there is very little overlap in the characteristics and behaviour of different transforms. As such, jVega Transforms are *not* implemented via inheritance. Instead `Transform` objects implement a common `Transform` interface. The Transform interface does not itself impose any behaviour, but instead acts as a 'Marker' interface to allow compile-time type checking and polymorphism [48, p.179].

```
1  Transform filter = simpleFilter("datum['Horsepower'] != null");
2  Transform kilos = simpleFormula("datum['Weight_in_lbs'] * 0.454","kilos");
3
4  VegaSpec transformExample = new VegaSpec.BuildSpec()
5      .setDescription("Transform Example")
6      .setNewDataset(new VegaDataset.BuildDataset()
7          .withName("source")
8          .withUrl("data/cars.json")
9          .withTransform(filter)
10         .withTransform(kilos)
11         .build())
12     .createVegaSpec();
```

Figure 4.8: *Example code showing two simple transforms on a Vega dataset, filtering the dataset to exclude nulls, and adding a field using a calculation. The transforms are static imports from the jVega FormulaTransform and FilterTransform classes.*

```
{
  "description": "Transform Example",
  "data": [
    {
      "name": "source",
      "url": "data/cars.json",
      "transform": [
        {
          "type": "filter",
          "expr": "datum['Horsepower'] != null"
        },
        {
          "type": "formula",
          "expr": "datum['Weight_in_lbs'] * 0.454",
          "as": "kilos"
        }
      ]
    }
  ]
}
```

Figure 4.9: *The resulting specification produced from the code in Figure 4.8, after serialization with jVega's toJson() method.*

4.4.3 Vega Marks, Axes and Encodings

The remaining jVega classes are shown in Figure 4.10, providing builders to specify the visual elements of Vega visualisations and their encoding.

`Encodings` are implemented via inheritance, in the same way as `Scales`, to allow the sharing of common attributes and behaviours – but giving each type of `Encoding` distinct attributes and behaviours, as well as clear type checking.

The ValueRef class offers a number of factory methods to make the specification of Vega values more concise - whether these are constants, variables or scale domains. [49].

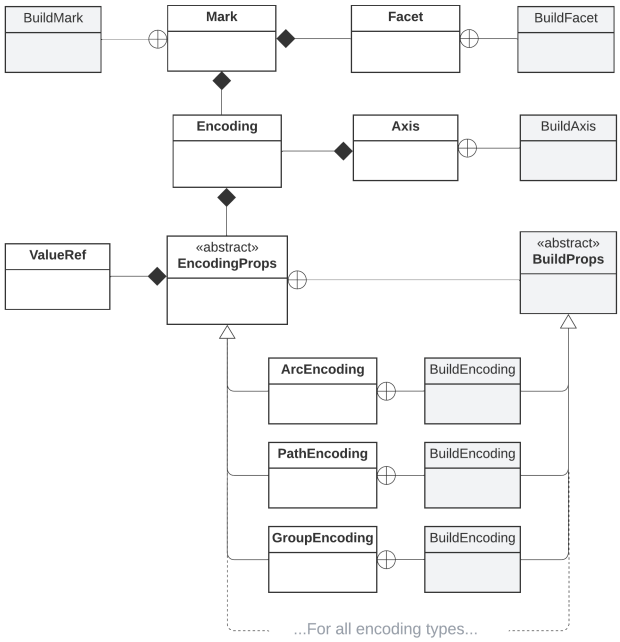


Figure 4.10: *A simplified UML diagram showing the classes provided for the building of marks and axes.*

```
VegaSpec marksExample = new VegaSpec.BuildSpec()
  .setDescription("Marks example")
  .setNewMark(new Mark.BuildMark()
      .withType("rect")
      .withDataSource("rawData")
      .withEnter(new RectEncoding.BuildEncoding()
          .withX(ValueRef.ScaleField("xscale", "barLabel"))
          .withWidth(ValueRef.ScaleBand("xscale", 1))
          .withY(ValueRef.ScaleField("yscale", "barHeight"))
          .withY2(ValueRef.ScaleValue("yscale", 0))
          .build())
      .withUpdate(new RectEncoding.BuildEncoding()
          .withFill(ValueRef.Value("steelblue")).build())
      .withHover(new RectEncoding.BuildEncoding()
          .withFill(ValueRef.Value("red")).build())
      .build())
  .createVegaSpec();
```

Figure 4.11: *An example of a 'rect' mark, with colour highlighting on hover, specified in jVega.*

As shown by comparison between the code in Figure 4.11 and the resulting Vega specification in Figure 4.12, this design abstracts a considerable amount of the nesting in the specification of Vega mark encodings - allowing developers to interact more simply through `withEnter`, `withUpdate` and `withHover` builder methods.

```
{
  "description" : "Marks example",
  "data": [{
    "name": "rawData",
    "url": "data/cars.json"
  }],
  "marks" : [ {
    "type" : "rect",
    "from" : {
                "data" : "rawData"
        },
    "encode" : {
      "enter" : {
        "_type" : "rect",
        "x" : [ {
          "field" : "barLabel", "scale" : "xscale"
        } ],
        "width" : [ {
          "scale" : "xscale", "band" : 1
        } ],
        "y" : [ {
          "field" : "barHeight", "scale" : "yscale"
        } ],
        "y2" : [ {
          "value" : 0, "scale" : "yscale"
        } ]
      },
      "update" : {
        "_type" : "rect",
        "fill" : [ {
          "value" : "steelblue"
        } ]
      },
      "hover" : {
        "_type" : "rect",
        "fill" : [ {
          "value" : "red"
        } ]
      }
    }
  } ]
}
```

Figure 4.12: *The resulting specification produced from the code in Figure 4.11, after serialization with jVega's toJson() method.*

4.5 jVega Evaluation

4.5.1 Implementation Benefits

The jVega code shown in Figures 4.3, 4.6, 4.8 and 4.11 when compared to the JSON specifications produced in Figures 4.4, 4.7, 4.9 and 4.12, demonstrates the achievement of the design goals set out in Section 4.3. These comparisons also demonstrate a number of broader contributions to the Vega ecosystem, namely:

Encouraging clean, readable code

jVega considerably reduces the number of nested layers required to specify deeply-nested JSON objects. This is achieved by abstracting the construction of objects and their children through the provided builder and factory methods.

The population of certain attributes (such as scale 'type' fields) can also be inferred from Java typing, whilst very common properties are specified as jVega defaults, which significantly reduces verbose JSON to a more compact Java specification.

Figure 4.13 also demonstrates the advantage of a wrapper for Vega specification, in allowing elements of the specification to be defined as variables (e.g. *scaleName*). In complex specifications where components are cross-referenced throughout, this promotes clean, robust code and allows programmatic variation of visualisation components.

Type checking and code completion

The sub-classing approach provides guidance to developers on what objects and attributes are valid at any point in the construction of a specification. Figure 4.13 demonstrates the code completion prompts that are available when constructing a band scale, for example, showing the validity of the 'align' parameter, but not the 'zero' parameter, which can only apply to quantitative scales.

Designing for extensibility and scale

As the Vega specification evolves – perhaps involving the addition or update of scale types and their properties – the modular, object-orientated design will allow easier maintenance of the software.

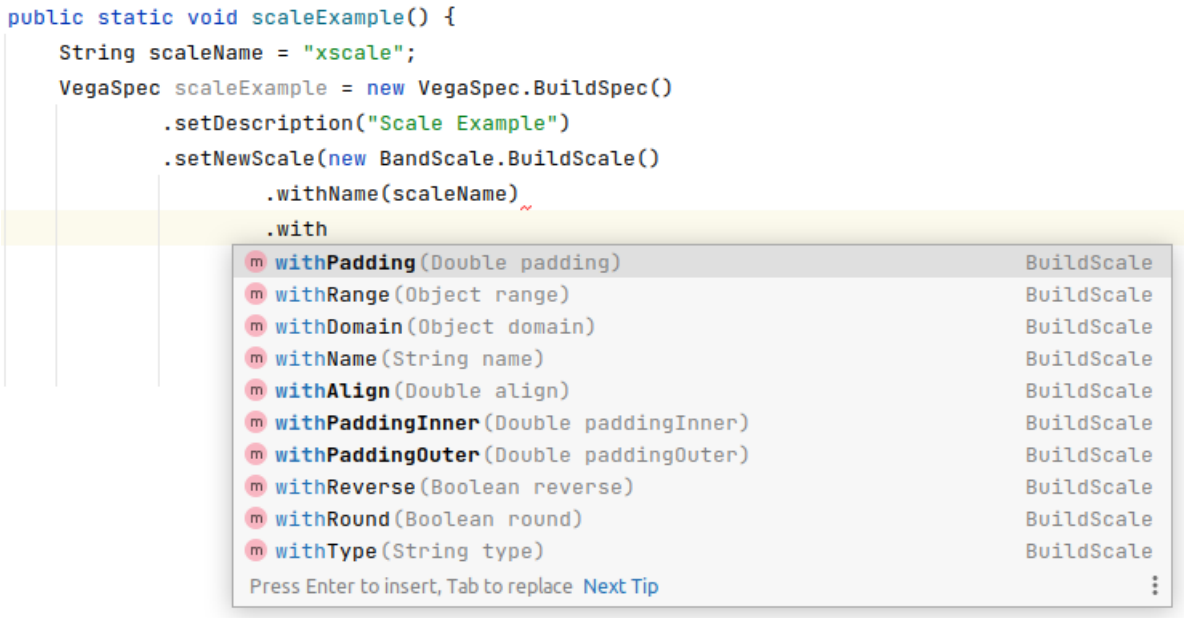


Figure 4.13: *The code completion prompts available in IntelliJ IDEA when constructing a band scale. These are limited to valid band scale attributes only.*

4.5.2 Limitations

Limited support for responsive design

The primary limitation of this approach – which was the main reason that Vega was not used more extensively throughout this project – is the lack of support for responsive design for web visualisation. Hard-coded visualisation dimensions within the specification make a responsive front-end much more difficult to achieve. Whilst this is in part a limitation of the Vega ecosystem itself, jVega could be developed further to support more dynamic specification in the future.

Complexity of full-stack design

Though there are clear performance benefits in performing computations for visualisation specification in back-end Java logic, this also introduces complexity in the need to re-compute visualisation specifications in response to front-end events and so generates much more traffic between client and server. This ultimately represents a design decision, which will be dependent on specific application requirements.

Application Evaluation

5.1 Formal Evaluation Criteria

The key evaluation criteria for visualisation recommendation tools set out in the literature are 'effectiveness' and 'expressiveness' – referring to the ability to fully express the facts of a dataset, without encoding incorrect or extraneous information, and employing the correct channels for representation (e.g. colour, size and ordering) to ensure correct visual perception of the information conveyed.[1]

The application achieves these goals by virtue of the underlying approach set out by McBrien and Poulovassilis [1]. By mapping data characteristics and relationships directly to visualisation schema patterns, the approach ensures that visualisation recommendation takes into consideration data types (e.g. discrete vs. continuous data types) and relationships between data (e.g. hierarchical vs. non-hierarchical relationships), as well as ensuring that dimensions of the data are deliberately mapped to specific dimensions of visualisations.

A good demonstration of this ability comes in the visualisation of weak entity relationships.

In the Mondial database, for example, *spoken* is a weak entity, with the *country* entity acting as its strong entity. In the developed application, a visualisation of the number of speakers of languages by country will produce a recommendation for a grouped or stacked bar chart, explicitly mapping languages to the colour channel, grouping them by country, and demonstrating the number of speakers through the size channel. This is a demonstration of the application meeting 'expressiveness' criteria.

[1] These concepts are also discussed in Section 2.2.

A visualisation of the same fields significantly will *not* produce a recommendation for a line chart, as the spoken language field does not hold an inherent order. By contrast a visualisation of province, population and year will produce a line chart as the year field is identified as being a continuous field which can be ordered. This is a demonstration of the application meeting 'effectiveness' criteria.

The key area in which the 'effectiveness' and 'expressiveness' of recommended visualisations could be improved is in the treatment of colour encoding. Though the application does encode variables to colour channels, this is largely achieved in the visualisation layer (for example, in assigning a colour legend to a treemap). Colour fields are not mapped specifically during creation of the visualisation schema, which would make this channel more visible to the user, and could allow assignment of the colour channel to be adjusted if necessary.

5.2 Functionality Testing

A set of visualisation tasks put together by other members of the department at Imperial College London was used to test the effectiveness of the application in catering to real, unseen analytical questions. These tests were run after the main development phase and used to fine tune the application, as well as informing areas for future development.

Table 5.1 below summarises the tests run, and the performance of the application in these tests.

The summary makes a distinction between tasks that the application can solve via pure 'point-and-click' functionality as opposed to visualisation tasks that require 'custom SQL' and which may therefore be more suited to expert users.

Test	Result	Method	Notes
The current population of countries in Europe	Pass	Point & click	Produces a basic entity bar chart
The relationship between GDP and unemployment for all countries	Pass	Point & click	Recognises 'economy' as a subset of 'country' - recommends a scatter chart
The length of the borders for all countries in South America	Pass	Point & click	Recognises a reflexive many-many relationship - recommends a chord diagram
The top ten Mexican airports by elevation	Pass	Custom SQL	Produces a basic entity bar chart
The GMT offset of all countries --- All country-city pairs where the population of the city has been at least 5,000,000 at some point	Fail --- Fail	n/a --- n/a	Requires chaining joins (country- province, province-city, city-airport) where the visualisation pattern is between the first and last entity in the chain
Countries that are in two continents	Pass	Custom SQL	Recognises a many-many relationship - recommends a sankey diagram
The relationship between industry and service GDP in the economies of different countries	Pass	Point & click	Recognises the 'economy' entity has an 'is-a' relationship with country - recommends a scatter chart
Display the the population of France, Spain, and Italy for the years 1960 to 2015	Pass	Custom SQL	Includes a line chart in recommendations given the numerical 'year' key
Display the history of the population level of each country in Oceania	Pass	Custom SQL	Continent can only be added to the selection as part of custom SQL
Display the elevation of all airports that are at least 2,000 in altitude	Pass	Custom SQL	Visualisation must be done using iata_code, as airport name is not unique
Display how the population of continents is divided between countries	Pass	Custom SQL	Recognises both one-many and many-many relationships depending on the subset of data.
The languages spoken in South American countries, excluding languages spoken by less than 1,000,000 people on the continent	Pass	Custom SQL	Requires considerable updates to the generated SQL after field selection

Table 5.1: *A table showing the results of functional tests*

Of the 13 visualisation tasks set by members of the department, the application was able to correctly visualise 11 out of 13 tasks. Of these, 4 were achievable through the simple point-and-click interface – with the remaining 7 requiring customisation of the generated SQL queries to varying levels of complexity.

These results are promising in that the tasks represent a good range of visualisation tasks from a number of independent users and the application is able to support more than 80% of these.

An area for further consideration is that many of these tasks required the use of custom SQL, and therefore may be more accesible to expert users. These results may be skewed, however, given that many of the tasks were intentionally designed for complexity in order to stretch the application.

On that basis, evaluation of accessibility for different user groups were mainly evaluated through the expert and non-expert test groups – which are discussed below.

5.3 User feedback

5.3.1 Expert Group Feedback

A group of professional analysts were shown the tool and asked for their feedback. The analysts' job titles ranged from *Analyst*, to *Lead Analyst* and *Head of Analytics*, in order to gather feedback from professionals with a range of experience levels and analytical goals.

The evaluation was treated as qualitative focus group – with participants asked to give verbal feedback during demonstration of the tool and whilst trying out the tool themselves.

The main areas of feedback were:

Support for more complex visualisations is a welcome feature, though reserved for special cases

The expert group agreed that support for more advanced visualisations such as sankey and chord diagrams was a useful feature – with many having spent considerable amounts of time trying to build these as custom visualisations in tools such as Tableau, with unsatisfactory results.

Whilst some had used tools such as D3 to produce these visualisations it was generally agreed that it is rare for teams to possess the JavaScript knowledge or to have the necessary infrastructure to serve JavaScript visualisations to users. Whilst it was agreed that this was a very useful feature – there was generally agreement that the need for these visualisations was rare, though of high value when appropriate.

Automated query generation helps to speed up an exploratory workflow

There was also consensus that the ability to automate the initial build of queries would be very helpful in speeding up earlier exploratory stages in the analytical workflow – particularly in large business systems where the database structure may not be well known to the analyst.

It was also emphasised, though, that in the majority of analytical use cases the query would need to be refined and extended to reflect more complex subjects. The fact that query generation currently only supports pairs of entities was flagged as a limitation in this regard.

Visualisation best practice guidance is beneficial for business users

More senior analysts in particular supported the clear signposting of recommended vs not-recommended visualisations, as they felt that this would help to reinforce best practices in organisations where there is a drive towards data democratisation and encouraging business users to interact directly with organisational data. Their feedback was that in existing tools – such as Tableau, Power BI and Excel — recommended visualisations were either more subtle or non-existent, leading to ineffective or misleading representations of data by less experienced users.

The design of the tool relies on well maintained data systems

Many highlighted their experience that data systems are often not well maintained in their organisation in terms of the presence of database constraints – particularly foreign keys.

This would present a challenge for the current functioning of the tool, although some more senior analysts hypothesised that having tools such as this with clear business benefits could provide an incentive for the better maintenance of constraints in business databases.

Nonetheless this is a current limitation, which would need to be addressed in future iterations of the tool – for example by supporting reverse engineering of database constraints where they are not formally defined.

More support for a SQL-first workflow would be beneficial

Despite an appreciation for the benefits of automated SQL generation from the point-and-click interface, many analysts suggested that the option for a pure SQL workflow would be beneficial, in terms of allowing the process to start with SQL specification, rather than field selection.

This is discussed under future work, as further development work would be required, particularly in terms of SQL query parsing to extract visualisation schema information from SQL queries and result sets.

More support for data-type differentiation and data aggregation would be beneficial

The analytical users were in agreement that key features for future developments should be greater control over data types (for example in allowing users to specify temporal or geographical data fields) as well as support for recognising visualisation patterns in aggregated data sets (e.g. where unique fields are created through grouping and aggregation)

5.3.2 Non-expert Group Feedback

Guided visualisation recommendations are preferential to tools such as Excel

Non-analysts were impressed by the ability of the tool to create high-quality, interactive visualisations with minimal user input. Compared to the tools that these users were familiar with, namely Microsoft Excel, there was also an appreciation of the clear recommendation of visualisations – rather than leaving the choice of visualisation entirely down to the user, which the participants expressed a lack of confidence in, and felt to be a time-consuming process.

The presence of non-recommended visualisations is a useful tool for learning

This user group also appreciated the ability to see the results of 'non-recommended' visualisations, as they felt this helped to cement the reasoning behind the choice of recommended visualisations. Users were in agreement that these should still be visible to users – as a more opinionated approach (in terms of hiding these visualisation options completely) would be too constrained and limit options for

exploration and learning.

Tabular representations of the source data would be helpful for initial understanding

Non-expert users in particular found it difficult to understand the underlying datasets without access to tabular representations of the data. This may be in part due to a lack of familiarity with databases and more exposure to spreadsheets. But users felt this could be largely resolved through access to tabular previews of each entity or relationship.

5.3.3 Shared feedback

The reason for unmatched visualisation schema patterns can be unclear

All users expressed a desire to get more feedback from the application when no visualisation schema was matched. For example, the application will not visualise elevation by airport name, because the airport name field is not actually unique (without additional information such as the city it is located in). Though this information can be inferred from the 'schema' page of the application – from the fact that the name field is not mapped as a key – users felt that in this scenario more guidance would be beneficial to users.

5.4 Ethical Considerations

5.4.1 Data Privacy, Security and Retention

When the application is made available to users to analyse their own private databases, it will be critical for the application to ensure proper data security, to avoid data subjects having their data accessed by unauthorised parties. To ensure that potentially sensitive information is kept safe the application should ensure the use of strong encryption protocols for the data at rest and in transit.

Given that the application does not yet support analysis of private user databases, security features have not been prioritised. The application has still been developed with best practice in mind, but a security audit would be necessary before handling user data systems.

Equally, given that visualisation schemas are stored in the application database, any handling of data from user databases would need to consider requirements regarding data protection and retention under General Data Protection Regulation (GDPR) [50]. For example, a mechanism would be needed to delete personal data that is no longer required, and to set retention periods that avoid indeterminate retention of personal information.

5.4.2 Transparency

Given that the application is designed to help analysts interpret information, there is a responsibility to provide methodological transparency to avoid injecting unseen bias into analytical processes. As far as possible this has been achieved through the in-depth methodology provided in this document, as well as features of the user interface that provide information on how selected fields have been mapped to visualisations. Allowing users to view visualisations that are not recommended also avoids the risk that the application will incorrectly direct analytical enquiries.

Future Work

6.1 Implementing a SQL-first user journey

One of the main requested features in user feedback was a SQL-first user journey – in addition to the point-and-click interface that currently exists. This would allow analysts who are more familiar with business systems to begin analysis with more complex SQL queries.

To support this user journey it will be necessary to devise an approach to parsing SQL queries and extracting visualisation schema patterns from the query string.

This will involve some additional complexity as the approach would need to consider how to differentiate entities and relationships that define the visualisation schema pattern from those that are included for other purposes (such as filtering), as well as differentiating attributes that play an active role in the visualisation schema pattern from those included for context.

As part of these developments, broader support for different SQL variants would also be required, as the current implementation is built primarily for PostgreSQL, and operates primarily on the assumption of standard SQL syntax and semantics.

6.2 Support for additional visualisation schema patterns

Functional testing of the application revealed additional visualisation schema patterns that the application is not currently able to support.

In particular, hierarchical patterns involving multiple layers such as that shown in Figure 6.1 are a common occurrence, meaning that support for this pattern would be a valuable feature to prioritise in future development.

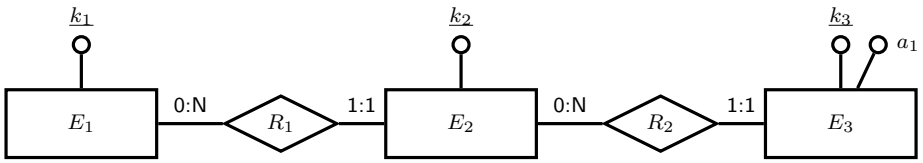


Figure 6.1: *A hierarchical schema pattern for three entities*

Patterns such as this should result in multi-layered hierarchical visualisations – for example, a treemap representing the relationship of country, province and city populations.

Another priority area for future expansion of supported visualisation schema patterns would be the implementation of user-defined data types. For example, the ability for users to supplement SQL data types with broader data categories such as 'geographical' data, would allow the recommendation of further visualisation types such as maps.

6.3 Support for user databases and persisted user profiles

User testing revealed a desire to test the application on a wider range of databases. To this end features could be developed to support connection to private user databases, as well as the ability of users to save their sessions in a user profile to return to their analysis at a later date.

The application is already built in a way that would support these developments, with the ability to produce database schema profiles from database and user information and to persist these profiles in MongoDB for use at a later date. Persistence in MongoDB would also support functionality for saving and resuming user sessions.

The key elements that would need to be implemented to fully support this functionality would be efficiently managing connection pools to a larger number of user databases, as well as securely storing user database credentials.

6.4 Expanding support for Vega visualisation specification

Though jVega classes are able to support the specification of full and complex Vega visualisations, the application was not able to make full use of this functionality due to limitations in the responsiveness of Vega visualisations.

Further developments to jVega to resolve these issues would allow work to resume in building Vega templates for use in the application - to support a wider range of visualisations. A key development that would facilitate this in the short term would be to include the ultimate visualisation canvas size as a parameter to jVega classes, to allow internal calculations to dynamically calculate sizing for graphical elements of the visualisation.

Appendices

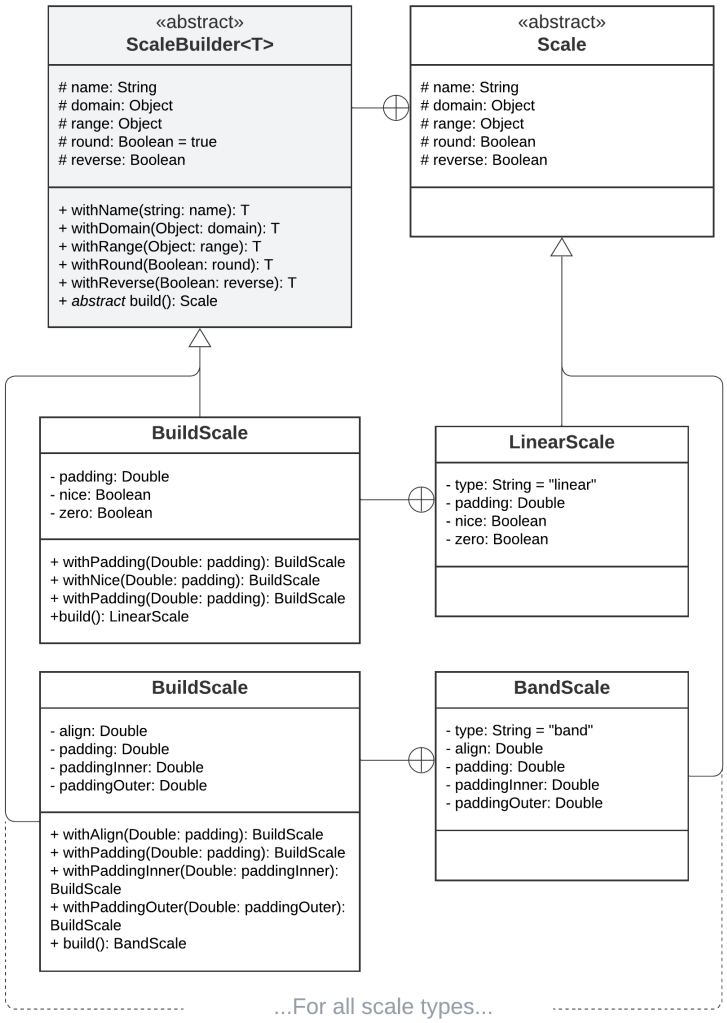


Figure 7.1: *A UML diagram showing fields and methods for jVega Scale classes*

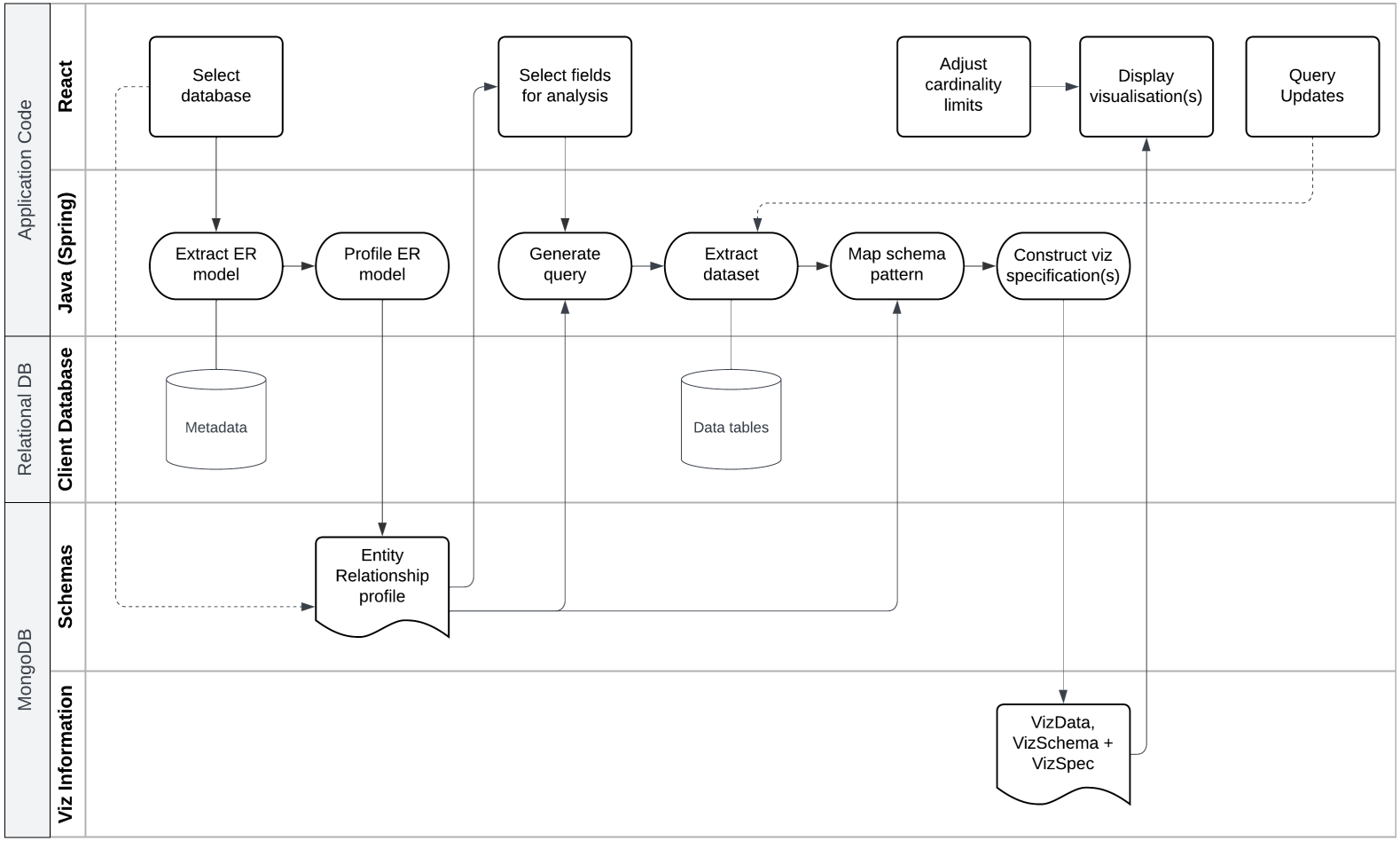


Figure 7.2: *The end-to-end design of the application*

Bibliography

[1] Peter McBrien and Alexandra Poulovassilis. Towards data visualisation based on conceptual modelling. In Juan C. Trujillo, Karen C. Davis, Xiaoyong Du, Zhanhuai Li, Tok Wang Ling, Guoliang Li, and Mong Li Lee, editors, *Conceptual Modeling*, pages 91–99, Cham, 2018. Springer International Publishing.

[2] Kanit Wongsuphasawat, Zening Qu, Dominik Moritz, Riley Chang, Felix Ouk, Anushka Anand, Jock Mackinlay, Bill Howe, and Jeffrey Heer. Voyager 2: Augmenting visual analysis with partial view specifications. In *ACM Human Factors in Computing Systems (CHI)*, 2017. URL `http://idl.cs.washington.edu/papers/voyager2`.

[3] Graham Wills and Leland Wilkinson. Autovis: Automatic visualization. *Information Visualization*, 9(1):47–69, 2010. doi: 10.1057/ivs.2008.27. URL `https://doi.org/10.1057/ivs.2008.27`.

[4] Manasi Vartak, Samuel Madden, Aditya Parameswaran, and Neoklis Polyzotis. Seedb: Automatically generating query visualizations. *Proc. VLDB Endow.*, 7(13):1581–1584, aug 2014. ISSN 2150-8097. doi: 10.14778/2733004.2733035. URL `https://doi.org/10.14778/2733004.2733035`.

[5] Steven F. Roth, John Kolojejchick, Joseph Mattis, and Mei Chuah. Sagetools: An intelligent environment for sketching, browsing, and customizing data graphics. In *Proceedings of SIGCHI Conference Companion of the Conference on Human Factors in Computing Systems (CHI '95)*, pages 409 – 410, May 1995.

[6] Kanit Wongsuphasawat, Dominik Moritz, Anushka Anand, Jock Mackinlay, Bill Howe, and Jeffrey Heer. Towards a general-purpose query language for visualization recommendation. pages 1–6, 06 2016. doi: 10.1145/2939502.2939506.

[7] Jock Mackinlay, Pat Hanrahan, and Chris Stolte. Show me: Automatic presentation for visual analysis. *IEEE transactions on visualization and computer graphics*, 13:1137–44, 11 2007. doi: 10.1109/TVCG.2007.70594.

[8] Leland Wilkinson. *The Grammar of Graphics (Statistics and Computing)*. Springer-Verlag, Berlin, Heidelberg, 2005. ISBN 0387245448.

[9] Kanit Wongsuphasawat, Dominik Moritz, Anushka Anand, Jock Mackinlay, Bill Howe, and Jeffrey Heer. Voyager: Exploratory analysis via faceted browsing of visualization recommendations. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):649–658, 2016. doi: 10.1109/TVCG.2015.2467191.

[10] C. Stolte, D. Tang, and P. Hanrahan. Polaris: a system for query, analysis, and visualization of multidimensional relational databases. *IEEE Transactions on Visualization and Computer Graphics*, 8(1):52–65, 2002. doi: 10.1109/2945.981851.

[11] Ed Huai-Hsin Chi and J.T. Riedl. An operator interaction framework for visualization systems. In *Proceedings IEEE Symposium on Information Visualization (Cat. No.98TB100258)*, pages 63–70, 1998. doi: 10.1109/INFVIS.1998.729560.

[12] S. S. Stevens. On the theory of scales of measurement. *Science*, 103(2684):677–680, 1946. doi: 10.1126/science.103.2684.677. URL `https://www.science.org/doi/abs/10.1126/science.103.2684.677`.

[13] Jacques Bertin. *Semiology of Graphics*. University of Wisconsin Press, 1983. ISBN 0299090604.

[14] William S. Cleveland and Robert McGill. Graphical perception: Theory, experimentation, and application to the development of graphical methods. *Journal of the American Statistical Association*, 79(387):531–554, 1984.

[15] Colin Ware. *Information Visualization: Perception for Design: Second Edition*. 04 2004.

[16] Hadley Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. ISBN 978-3-319-24277-4. URL `https://ggplot2.tidyverse.org`.

[17] Hadley Wickham. ggplot2. *Wiley Interdisciplinary Reviews: Computational Statistics*, 3:180 – 185, 02 2011. doi: 10.1002/wics.147.

[18] Hadley Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1):3–28, 2010. doi: 10.1198/jcgs.2009.07098.

[19] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017. doi: 10.1109/TVCG.2016.2599030.

[20] Arvind Satyanarayan, Kanit Wongsuphasawat, and Jeffrey Heer. Declarative interaction design for data visualization. *UIST 2014 - Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology*, pages 669–678, 10 2014. doi: 10.1145/2642918.2647360.

[21] Michael Bostock, Vadim Ogievetsky, and Jeffrey Heer. D3: Data-driven documents. *IEEE Trans. Visualization & Comp. Graphics (Proc. InfoVis)*, 2011. URL `http://vis.stanford.edu/papers/d3`.

[22] Vega – a visualization grammar, . URL `https://vega.github.io/vega/`.

[23] Vega-lite – a high level grammar of interactive graphics, . URL `https://vega.github.io/vega-lite/`.

[24] Jock D. Mackinlay. Automating the design of graphical presentations of relational information. *ACM Trans. Graph.*, 5:110–141, 1986.

[25] Manasi Vartak, Sajjadur Rahman, Samuel Madden, Aditya Parameswaran, and Neoklis Polyzotis. Seedb: Efficient data-driven visualization recommendations to support visual analytics. *Proc. VLDB Endow.*, 8(13):2182–2193, sep 2015. ISSN 2150-8097. doi: 10.14778/2831360.2831371. URL `https://doi.org/10.14778/2831360.2831371`.

[26] Leland Wilkinson, A. Anand, and Robert Grossman. Graph-theoretic scagnostics. volume 5, pages 157– 164, 11 2005. ISBN 0-7803-9464-X. doi: 10.1109/INFVIS.2005.1532142.

[27] Wolfgang May. Information extraction and integration with florid: The mondial case study. 1999. URL `https://api.semanticscholar.org/CorpusID:59783661`.

[28] Jean-Marc Petit, Farouk Toumani, Jean-Francois Boulicaut, and Jacques Kouloumdjian. Towards the reverse engineering of denormalized relational databases. In *Proceedings of the Twelfth International Conference on Data Engineering*, ICDE '96, page 218–227, USA, 1996. IEEE Computer Society. ISBN 0818672404.

[29] Christian Soutou. Relational database reverse engineering: Algorithms to extract cardinality constraints. *Data Knowledge Engineering*, 28(2):161–207, 1998. ISSN 0169-023X. doi: https://doi.org/10.1016/S0169-023X(98)00017-2. URL `https://www.sciencedirect.com/science/article/pii/S0169023X98000172`.

[30] William J. Premerlani and Michael R. Blaha. An approach for reverse engineering of relational databases. *[1993] Proceedings Working Conference on Reverse Engineering*, pages 151–160, 1993.

[31] Martin Andersson. Extracting an entity relationship schema from a relational database through reverse engineering. *International Journal of Cooperative Information Systems*, 04(02n03):259–285, 1995. doi: 10.1142/S0218843095000111. URL https://doi.org/10.1142/S0218843095000111.

[32] Stefan Reljic, Drazen Brdjanin, and Goran Banjac. Reverse engineering of relational database schema based on universal metadata queries. pages 1–6, 03 2022. doi: 10.1109/INFOTEH53737.2022.9751287.

[33] Spring data mongodb. URL https://docs.spring.io/spring-data/mongodb/docs/current/reference/html/.

[34] Amazing-er github repository, . URL https://github.com/BoanZhu/ER-Backend/.

[35] Amazing-er javadoc, . URL https://www.javadoc.io/doc/io.github.MigadaTang/amazing-er/latest/index.html.

[36] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional Computing Series. Pearson Education, 1994. ISBN 9780321700698. URL https://books.google.co.uk/books?id=6oHuKQe3TjQC.

[37] Mdn: Set documentation. URL https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set.

[38] Ant-v: G visualisation engine, . URL https://g-next.antv.vision/en/.

[39] Ant-v: G6 network visualisation library, . URL https://g6.antv.antgroup.com/en/.

[40] Ant-v: L7 geo-spatial visualisation library, . URL https://l7.antv.antgroup.com/en/.

[41] Ant-v: F2 mobile visualisation library, . URL https://f2.antv.antgroup.com/en/examples.

[42] Ant-v: Homepage, . URL https://antv.antgroup.com/en/.

[43] Ant-v: Ant design charts library, . URL https://charts.ant.design/en/.

[44] About the vega project, . URL https://vega.github.io/vega/about/.

[45] Vega-lite bindings for programming languages, . URL https://vega.github.io/vega-lite/ecosystem.html#bindings-for-programming-languages.

[46] Vega documentation: Scales, . URL https://vega.github.io/vega/docs/scales/.

[47] Vega documentation: Transforms, . URL https://vega.github.io/vega/docs/transforms/.

[48] Joshua Bloch. *Effective Java*. Addison-Wesley, Boston, MA, 2018. ISBN 978-0-13-468599-1. URL https://www.safaribooksonline.com/library/view/effective-java-third/9780134686097/.

[49] Vega documentation: Value references, . URL https://vega.github.io/vega/docs/types/#Value.

[50] Information commissioner's office: Gdpr resources. URL https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/.